Architettura dei Calcolatori

Appunti del corso · a.a. 2026/27 · Mobius

Appunti personali e non ufficiali, rivisti e confrontati con il programma ufficiale del corso. Non sostituiscono le lezioni né i testi consigliati.

Indice

1. Astrazione, prestazioni e l’ISA RISC-V
2. Programmazione in assembly RISC-V
3. L’ALU e l’aritmetica del calcolatore
4. Il processore, la pipeline e il parallelismo fra istruzioni
5. La gerarchia di memoria, la cache e l’input/output

1 · Astrazione, prestazioni e l’ISA RISC-V

In questo modulo. Servono le reti logiche (porte, registri, rappresentazione dei numeri in complemento a 2). Si parte dall’idea che regge tutta la materia, i livelli di astrazione con le loro interfacce, e dal modello di von Neumann. Poi si impara a misurare le prestazioni di un calcolatore con l’equazione del tempo di CPU e con la legge di Amdahl. La seconda parte presenta l’insieme di istruzioni (ISA) del processore RISC-V: registri, memoria, formati di codifica, istruzioni aritmetiche, di accesso alla memoria e di salto. Alla fine del modulo si sa confrontare due macchine con l’equazione delle prestazioni, e codificare e decodificare a mano un’istruzione RISC-V.

> **Aggiunta in revisione · fonte: slide del corso (Introduzione, RISC-V Instruction Set Architecture), articolo di Smith e Nair “The Architecture of Virtual Machines” (*IEEE Computer*, 2005), rapporto di von Neumann sull’EDVAC (1945); Patterson e Hennessy, *Struttura e progetto dei calcolatori*, edizione RISC-V (ADC-01, ADC-02)**
>
> Gli appunti a mano del corso coprono solo il datapath con pipeline (modulo 4): questo modulo è scritto in revisione dal materiale del corso.

1.1 Livelli di astrazione e interfacce

Un calcolatore moderno è troppo complesso per essere pensato tutto insieme. Lo si divide in **livelli**, ognuno dei quali offre a quello sopra un insieme di servizi attraverso un’**interfaccia** ben definita e nasconde i dettagli di come li realizza. Dal basso verso l’alto: i transistori e le porte logiche; le reti (sommatori, registri, memorie); la **microarchitettura**, cioè l’organizzazione dei blocchi che eseguono le istruzioni; l’**insieme di istruzioni** (ISA, *instruction set architecture*), che è il confine fra hardware e software; il sistema operativo; i linguaggi e le applicazioni.

L’ISA è l’interfaccia più importante. Descrive tutto ciò che un programmatore in linguaggio macchina deve sapere: quali istruzioni esistono, come si codificano, quali registri ci sono, come si accede alla memoria. Non dice come le istruzioni vengono eseguite: due processori con la stessa ISA (per esempio due generazioni della stessa famiglia) eseguono gli stessi programmi anche se dentro sono molto diversi. È il motivo per cui il software sopravvive a molte generazioni di hardware.

Il prezzo delle interfacce è la rigidità: un programma compilato per un’ISA funziona solo sull’hardware che la realizza, e un sistema operativo solo sull’ISA per cui è scritto. La **virtualizzazione** supera questo limite inserendo uno strato di software che realizza un’interfaccia su un’altra. Una **macchina virtuale di processo** offre a un singolo programma un ambiente diverso da quello reale: la macchina virtuale Java, o un emulatore che esegue codice per un’altra ISA. Una **macchina virtuale di sistema** offre un’intera macchina, su cui gira un sistema operativo completo: è il caso degli *hypervisor* che fanno girare più sistemi operativi sullo stesso server.

1.2 Il modello di von Neumann

Quasi tutti i calcolatori seguono lo schema descritto da von Neumann nel 1945: un’**unità centrale** (CPU) con un’unità aritmetico-logica e un’unità di controllo, una **memoria** che contiene sia i dati sia le istruzioni, e i dispositivi di **ingresso e uscita**, collegati da un sistema di interconnessione (i bus). L’idea decisiva è il **programma memorizzato**: le istruzioni sono numeri come i dati, stanno nella stessa memoria, e la macchina si riprogramma cambiando il contenuto della memoria, non i collegamenti.

La CPU esegue le istruzioni con un ciclo che si ripete sempre uguale:

1. **prelievo** (*fetch*): legge dalla memoria l’istruzione all’indirizzo contenuto nel **program counter** (PC);
2. **decodifica**: capisce che istruzione è e legge gli operandi dai registri;
3. **esecuzione**: fa l’operazione nell’ALU, oppure calcola un indirizzo;
4. **accesso alla memoria**, se l’istruzione legge o scrive un dato;
5. **scrittura del risultato** in un registro, e aggiornamento del PC (all’istruzione successiva, o a quella di destinazione di un salto).

Il collegamento fra CPU e memoria è anche il limite del modello: ogni istruzione e ogni dato devono passare di lì, e la memoria è molto più lenta della CPU. È il cosiddetto “collo di bottiglia di von Neumann”, a cui rispondono le cache del modulo 5.

1.3 Le prestazioni

Per confrontare due calcolatori bisogna decidere che cosa misurare. Il **tempo di esecuzione** (o tempo di risposta) di un programma interessa all’utente; il **throughput**, cioè il lavoro svolto per unità di tempo, interessa a chi gestisce un server. Qui si guarda il tempo di CPU di un programma, e una macchina è “n volte più veloce” di un’altra se

$$\frac{\text{Prestazione}_A}{\text{Prestazione}_B} = \frac{T_B}{T_A} = n.$$

Il processore lavora al ritmo di un clock con periodo T_{clk} (e frequenza $f = 1/T_{clk}$). Ogni istruzione richiede un certo numero di cicli; la media su un programma si chiama **CPI** (*cicli per istruzione*). Allora vale l’**equazione delle prestazioni**:

$$T_{CPU} = N_{istr} \times CPI \times T_{clk} = \frac{N_{istr} \times CPI}{f}.$$

I tre fattori dipendono da cose diverse. N_{istr} dipende dal programma, dal compilatore e dall’ISA; il CPI dalla microarchitettura (e dal tipo di istruzioni usate); T_{clk} dalla tecnologia e dalla microarchitettura. Migliorare uno solo di essi peggiorando gli altri può non servire: è l’errore di giudicare un processore solo dalla frequenza.

Se le istruzioni si dividono in classi con CPI diversi, il CPI medio è la media pesata con le frazioni ϕ_i di istruzioni di ogni classe:

$$CPI = \sum_i \phi_i \, CPI_i.$$

> **Due compilatori**
>
> Un programma compilato con il compilatore 1 esegue 2 milioni di istruzioni di classe A (CPI 1), 1 milione di classe B (CPI 2) e 1 milione di classe C (CPI 3); con il compilatore 2, 4 milioni di A, 1 di B e 1 di C. Cicli: $2 + 2 + 3 = 7$ milioni contro $4 + 2 + 3 = 9$ milioni. CPI medio: $\frac{7}{4} = 1{,}75$ contro $\frac{9}{6} = 1{,}5$. Il secondo compilatore ha il CPI migliore ma produce più istruzioni, e il programma è più lento: a parità di clock conta il prodotto $N_{istr} \times CPI$, non uno dei due.

Per lo stesso motivo il **MIPS** (milioni di istruzioni al secondo) non è una buona misura: dipende dal programma e dall’ISA, e una macchina con istruzioni più semplici ne esegue di più senza essere più veloce.

> **Legge di Amdahl**
>
> Se un miglioramento accelera di un fattore k una parte del programma che occupa la frazione F del tempo di esecuzione, il tempo nuovo è
>
> $$T' = T\left((1 - F) + \frac{F}{k}\right), \qquad S = \frac{T}{T'} = \frac{1}{(1 - F) + F/k}.$$

La conseguenza è che l’accelerazione complessiva è limitata dalla parte non migliorata: anche con $k \to \infty$, $S \to \frac{1}{1-F}$. Se le moltiplicazioni occupano l’80% del tempo, nessun moltiplicatore, per quanto veloce, rende il programma più di 5 volte più rapido. La regola pratica è “rendere veloce il caso frequente”.

1.4 L’ISA RISC-V: registri e memoria

RISC-V è un’ISA aperta, progettata a Berkeley a partire dal 2010, di tipo **RISC** (*reduced instruction set computer*): poche istruzioni semplici, tutte della stessa lunghezza (32 bit nella versione base RV32I), con operazioni aritmetiche solo fra registri e accesso alla memoria solo con istruzioni di load e store (architettura

load/store). Le scelte seguono quattro principi di progetto: la semplicità favorisce la regolarità; più piccolo è più veloce; rendere veloce il caso frequente; un buon progetto richiede buoni compromessi.

Registri. Ci sono 32 registri interi da 32 bit, `x0` … `x31`. `x0` vale sempre 0: scriverci non ha effetto, e serve a ottenere costanti e operazioni utili senza istruzioni apposite. Gli altri hanno un nome "di convenzione" (ABI) che dice a che cosa servono nelle chiamate di procedura (modulo 2):

Registro	Nome	Uso
`x0`	`zero`	costante 0
`x1`	`ra`	indirizzo di ritorno
`x2`	`sp`	puntatore alla cima dello stack
`x3`, `x4`	`gp`, `tp`	puntatore globale, puntatore al thread
`x5`–`x7`, `x28`–`x31`	`t0`–`t6`	temporanei (non preservati dalle chiamate)
`x8`, `x9`, `x18`–`x27`	`s0`–`s11`	salvati (preservati dalle chiamate); `s0` è anche il frame pointer `fp`
`x10`–`x17`	`a0`–`a7`	argomenti; `a0` e `a1` anche valori di ritorno

Memoria. La memoria è un vettore di byte, ciascuno con il suo indirizzo (memoria **indirizzata al byte**). Una parola (*word*) è di 4 byte, e i dati di 4 byte di solito stanno a indirizzi multipli di 4 (**allineamento**). RISC-V è **little endian**: in una parola il byte meno significativo sta all'indirizzo più basso. La parola `0x12345678` scritta all'indirizzo 100 occupa i byte 100 (`0x78`), 101 (`0x56`), 102 (`0x34`), 103 (`0x12`).

1.5 I formati delle istruzioni

Tutte le istruzioni RV32I sono lunghe 32 bit e ricadono in sei formati. I campi che compaiono in più formati stanno sempre negli stessi bit, così la decodifica è semplice: l'**opcode** (bit 6-0) dice il formato e la famiglia di istruzioni, `rd` (bit 11-7) è il registro di destinazione, `rs1` (19-15) e `rs2` (24-20) sono i registri sorgente, `funct3` (14-12) e `funct7` (31-25) distinguono le istruzioni con lo stesso opcode.

Formato	31-25	24-20	19-15	14-12	11-7	6-0	Esempi
R	funct7	rs2	rs1	funct3	rd	opcode	`add`, `sub`, `and`, `slt`, `sll`
I	imm[11:0] (12 bit)		rs1	funct3	rd	opcode	`addi`, `lw`, `jalr`, `slli`
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	`sw`, `sb`
B	imm[12, 10:5]	rs2	rs1	funct3	imm[4:1, 11]	opcode	`beq`, `bne`, `blt`, `bge`
U	imm[31:12] (20 bit)				rd	opcode	`lui`, `auipc`
J	imm[20, 10:1, 11, 19:12]				rd	opcode	`jal`

Gli **immediati** sono costanti scritte nell'istruzione. Hanno 12 bit nei formati I, S e B, e si estendono con il segno a 32 bit prima dell'uso: un immediato I va da -2048 a 2047. Nel formato S l'immediato è spezzato in due pezzi per lasciare `rs2` al suo posto; nei formati B e J l'immediato è un **offset** rispetto al PC, in multipli di 2 byte (il bit 0 non si scrive perché vale sempre 0), e i suoi bit sono rimescolati per riusare gli stessi fili dell'hardware degli altri formati. Per costanti di 32 bit si combinano `lui` (carica i 20 bit alti) e `addi` (aggiunge i 12 bassi).

Codificare `add x9, x20, x21`

È di tipo R: opcode `0110011`, funct3 `000`, funct7 `0000000`. I registri sono `rd` = 9 (`01001`), `rs1` = 20 (`10100`), `rs2` = 21 (`10101`). Mettendo i campi in ordine, dal bit 31 al bit 0:

$$\underbrace{0000000}_{funct7}\ \underbrace{10101}_{rs2}\ \underbrace{10100}_{rs1}\ \underbrace{000}_{funct3}\ \underbrace{01001}_{rd}\ \underbrace{0110011}_{opcode}$$

cioè, a gruppi di quattro bit, `0000 0001 0101 1010 0000 0100 1011 0011` = `0x015A04B3`.

1.6 Le istruzioni principali

Aritmetiche e logiche. `add rd, rs1, rs2` ($rd = rs1 + rs2$), `sub`, `and`, `or`, `xor`; le versioni con immediato `addi`, `andi`, `ori`, `xori` (non esiste `subi`: basta `addi` con un immediato negativo). Gli shift `sll`, `srl` (logico, entra 0) e `sra` (aritmetico, entra il bit di segno), e `slli`, `srli`, `srai`: uno shift a sinistra di k moltiplica per 2^k. `slt rd, rs1, rs2` mette 1 in `rd` se $rs1 < rs2$ (con segno) e 0 altrimenti; `sltu` fa il confronto senza segno. Non c'è un registro di flag: gli esiti dei confronti finiscono in registri normali.

Accesso alla memoria. `lw rd, offset(rs1)` carica la parola all'indirizzo $rs1 + offset$; `sw rs2, offset(rs1)` la scrive. Per i byte: `lb` (estende con il segno), `lbu` (estende con zeri), `sb`; per le mezze parole `lh`, `lhu`, `sh`. L'unico modo di indirizzamento della memoria è registro base più spiazzamento.

Salti condizionati. `beq rs1, rs2, etichetta` salta se i registri sono uguali, `bne` se diversi, `blt` e `bge` (con segno), `bltu` e `bgeu` (senza segno). L'assemblatore traduce l'etichetta nell'offset rispetto al PC dell'istruzione di salto.

Salti incondizionati. `jal rd, etichetta` salva in `rd` l'indirizzo dell'istruzione successiva ($PC + 4$) e salta; `jalr rd, offset(rs1)` salta all'indirizzo $rs1 + offset$, salvando anch'esso $PC + 4$ in `rd`. Con `rd` = `ra` sono le istruzioni di chiamata di procedura; con `rd` = `x0` sono salti semplici.

Pseudo-istruzioni. L'assemblatore accetta istruzioni che non esistono in hardware e le traduce: `li rd, c` (carica una costante, con `addi` o `lui` più `addi`), `mv rd, rs` (`addi rd, rs, 0`), `nop` (`addi x0, x0, 0`), `j etichetta` (`jal x0, …`), `ret` (`jalr x0, 0(ra)`), `la rd, simbolo` (carica un indirizzo), `bgt`, `ble` (con gli operandi scambiati), `neg`, `not`.

Modi di indirizzamento. Riassumendo: *immediato* (l'operando è nell'istruzione: `addi`), *registro* (`add`), *base più spiazzamento* (`lw`, `sw`), *relativo al PC* (salti, `auipc`).

Esempi ed esercizi svolti

Aggiunta in revisione · fonte: esercizi nuovi sul modello delle slide del corso; codifiche controllate con `strumenti/rv.py` e conti con `strumenti/verifica_ampliamento.py` (ADC-01, ADC-02)

Esercizio 1. Codificare quattro istruzioni. Scrivere in esadecimale (a) `sub x5, x6, x7`, (b) `addi x5, x6, -12`, (c) `lw x10, 8(x2)`, (d) `sw x10, 12(x2)`.

a. Tipo R con funct7 `0100000` (è il bit che distingue `sub` da `add`), rs2 = 7, rs1 = 6, funct3 `000`, rd = 5, opcode `0110011`: `0x407302B3`.

b. Tipo I, opcode `0010011`, funct3 `000`. L'immediato -12 su 12 bit in complemento a 2 è `111111110100`. Con rs1 = 6 e rd = 5: `0xFF430293`. Le prime tre cifre esadecimali, `FF4`, sono proprio l'immediato.

c. Tipo I con opcode `0000011` e funct3 `010` (parola): immediato 8, rs1 = 2, rd = 10: `0x00812503`.

d. Tipo S, opcode `0100011`, funct3 `010`. L'immediato 12 = `000000001100` si spezza: imm[11:5] = `0000000` nei bit 31-25, imm[4:0] = `01100` nei bit 11-7. Con rs2 = 10 (il dato da scrivere) e rs1 = 2 (la base): `0x00A12623`.

Esercizio 2. Decodificare. Che istruzioni sono `0x40B50533` e `0x00C58593`?

`0x40B50533` = `0100 0000 1011 0101 0000 0101 0011 0011`. Opcode `0110011`: tipo R. funct3 = `000`, funct7 = `0100000`: è una `sub`. rd = bit 11-7 = `01010` = 10, rs1 = `01010` = 10, rs2 = `01011` = 11. È `sub x10, x10, x11`,

cioè `sub a0, a0, a1`.

`0x00C58593` = `0000 0000 1100 0101 1000 0101 1001 0011`. Opcode `0010011` con funct3 `000`: `addi`. rd = `01011` = 11, rs1 = `01011` = 11, immediato = `000000001100` = 12. È `addi a1, a1, 12`.

Esercizio 3. Il CPI medio e il confronto fra due macchine. Un programma ha il 50% di istruzioni ALU, il 20% di load, il 10% di store e il 20% di salti. Sulla macchina A i CPI sono 1, 5, 3, 2 e il clock è a 2 GHz; sulla macchina B sono 1, 2, 2, 2 con clock a 1,5 GHz. Quale macchina è più veloce, e di quanto?

$CPI_A = 0{,}5 + 1 + 0{,}3 + 0{,}4 = 2{,}2$; $CPI_B = 0{,}5 + 0{,}4 + 0{,}2 + 0{,}4 = 1{,}5$. Il numero di istruzioni è lo stesso (stessa ISA, stesso programma), quindi si confrontano i tempi per istruzione: $\frac{2{,}2}{2\ \text{GHz}} = 1{,}1$ ns per A, $\frac{1{,}5}{1{,}5\ \text{GHz}} = 1{,}0$ ns per B. B è più veloce di $\frac{1{,}1}{1{,}0} = 1{,}1$ volte, nonostante il clock più lento.

Esercizio 4. Amdahl. In un programma i calcoli in virgola mobile occupano il 40% del tempo. Conviene un'unità in virgola mobile 10 volte più veloce o un miglioramento che accelera tutto del 30%?

Con l'unità nuova: $S = \frac{1}{0{,}6 + 0{,}04} = 1{,}5625$. Con l'accelerazione generale, $S = 1{,}3$. Conviene la prima. Anche con un'unità infinitamente veloce, però, non si andrebbe oltre $\frac{1}{0{,}6} = 1{,}67$.

Esercizio 5. Costanti di 32 bit. Come si carica `0x12345678` in `t0`?

`lui t0, 0x12345` mette `0x12345000` in `t0`; `addi t0, t0, 0x678` aggiunge i 12 bit bassi. Funziona perché `0x678` ha il bit 11 a 0. Se i 12 bit bassi avessero il bit 11 a 1, `addi` li estenderebbe con il segno come numero negativo, e bisognerebbe caricare con `lui` la parte alta aumentata di 1: è il conto che fa l'assemblatore quando si scrive la pseudo-istruzione `li t0, 0x12345678`.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; soluzioni controllate con `strumenti/rv.py` e `strumenti/verifica_ampliamento.py` (ADC-01, ADC-02)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. Codificare un salto condizionato. Codificare `beq x5, x6, +16` (il salto va 16 byte avanti).

Passo 1: il formato. `beq` è di tipo B, opcode `1100011`, funct3 `000`.

Passo 2: l'immediato. L'offset 16 su 13 bit è `0 0000 0001 0000`: imm[12] = 0, imm[11] = 0, imm[10:5] = `000000`, imm[4:1] = `1000`.

Passo 3: i campi al loro posto. Bit 31 = imm[12] = 0; bit 30-25 = imm[10:5] = `000000`; rs2 = 6 = `00110`; rs1 = 5 = `00101`; funct3 = `000`; bit 11-8 = imm[4:1] = `1000`; bit 7 = imm[11] = 0; opcode `1100011`.

Passo 4: il risultato. `0000 0000 0110 0010 1000 1000 0110 0011` = `0x00628863`.

Guidato 2. Quanto si guadagna con un clock più veloce? Un programma impiega 10 s sulla macchina A con clock a 2 GHz. Si vuole una macchina B che lo esegua in 6 s; con un clock più veloce, però, B ha bisogno di 1,2 volte i cicli di A. Che frequenza serve?

Passo 1: i cicli di A. $\text{cicli}_A = T \times f = 10 \times 2 \cdot 10^9 = 2 \cdot 10^{10}$.

Passo 2: i cicli di B. $1{,}2 \times 2 \cdot 10^{10} = 2{,}4 \cdot 10^{10}$.

Passo 3: la frequenza. $f_B = \frac{2{,}4 \cdot 10^{10}}{6\ \text{s}} = 4$ GHz.

Passo 4: il controllo. $\frac{2{,}4 \cdot 10^{10}}{4 \cdot 10^9} = 6$ s. Raddoppiare la frequenza non basta a dimezzare il tempo, perché i cicli aumentano.

Guidato 3. Little endian. All'indirizzo 200 c'è la parola `0xA1B2C3D4`. Che cosa carica `lbu t0, 201(zero)`? E `lb t1, 200(zero)`?

Passo 1: i byte in memoria. Little endian: 200 → `D4`, 201 → `C3`, 202 → `B2`, 203 → `A1`.

Passo 2: `lbu`. Carica `0xC3` ed estende con zeri: `t0` = `0x000000C3` = 195.

Passo 3: `lb`. Carica `0xD4`, il cui bit 7 è 1, ed estende il segno: `t1` = `0xFFFFFFD4` = -44.

Riepilogo

Un calcolatore si organizza in livelli di astrazione separati da interfacce, la più importante delle quali è l'ISA, il confine fra hardware e software; la virtualizzazione realizza un'interfaccia sopra un'altra. Il modello di von Neumann mette dati e istruzioni nella stessa memoria e le esegue con il ciclo prelievo-decodifica-esecuzione. Il tempo di CPU è il prodotto di numero di istruzioni, CPI e periodo del clock; nessuno dei tre fattori da solo misura le prestazioni, e la legge di Amdahl limita il guadagno di ogni miglioramento parziale. RISC-V è un'ISA load/store con 32 registri (`x0` sempre nullo), memoria indirizzata al byte e little endian, e istruzioni da 32 bit in sei formati (R, I, S, B, U, J) con i campi dei registri sempre negli stessi bit; gli immediati si estendono con il segno, e gli offset dei salti sono relativi al PC.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (ADC-01, ADC-02)**

Prestazioni

$$T_{CPU} = N_{istr} \cdot CPI \cdot T_{clk}, \qquad CPI = \sum_i \phi_i CPI_i, \qquad \frac{\text{Prest}_A}{\text{Prest}_B} = \frac{T_B}{T_A}, \qquad S_{Amdahl} = \frac{1}{(1-F) + F/k} \leq \frac{1}{1-F}.$$

Formati RV32I

Formato	Campi (dal bit 31 al bit 0)
R	funct7 · rs2 · rs1 · funct3 · rd · opcode
I	imm[11:0] · rs1 · funct3 · rd · opcode
S	imm[11:5] · rs2 · rs1 · funct3 · imm[4:0] · opcode
B	imm[12, 10:5] · rs2 · rs1 · funct3 · imm[4:1, 11] · opcode
U	imm[31:12] · rd · opcode
J	imm[20, 10:1, 11, 19:12] · rd · opcode

Opcode più usati: `0110011` R, `0010011` I aritmetiche, `0000011` load, `0100011` store, `1100011` branch, `1101111` `jal`, `1100111` `jalr`, `0110111` `lui`.

Termini per l'orale

Termine	In una riga
ISA	insieme di istruzioni, registri e regole che un programma vede
Microarchitettura	organizzazione interna che realizza un'ISA
Macchina virtuale di processo / di sistema	ambiente per un programma / per un intero sistema operativo
Programma memorizzato	istruzioni e dati nella stessa memoria
CPI	numero medio di cicli per istruzione
RISC	ISA con poche istruzioni semplici e di lunghezza fissa
Architettura load/store	solo load e store accedono alla memoria
Little endian	il byte meno significativo all'indirizzo più basso
Estensione del segno	ripetere il bit di segno per allungare un numero
Pseudo-istruzione	istruzione dell'assemblatore tradotta in istruzioni reali

Domande che tornano spesso

- Che cos’è l’ISA e perché è un’interfaccia così importante? Che cosa sono le macchine virtuali?
- Descrivere il modello di von Neumann e il ciclo di esecuzione delle istruzioni.
- Ricavare l’equazione delle prestazioni; perché il MIPS è una misura fuorviante?
- Enunciare la legge di Amdahl e spiegarne le conseguenze.
- Descrivere i formati delle istruzioni RISC-V; perché i campi dei registri sono sempre negli stessi bit?
- Perché nei formati B e J l’immediato è “rimescolato” e non contiene il bit 0?

2 · Programmazione in assembly RISC-V

In questo modulo. Si usano i registri, i formati e le istruzioni del modulo 1 (*Astrazione, prestazioni e l'ISA RISC-V*), e un po' di C. Si vede come un compilatore traduce in assembly i costrutti del C: espressioni, condizioni, cicli, accessi ad array, matrici e stringhe. Poi si studia il modello di memoria di un programma (codice, dati statici, heap, stack) e il meccanismo delle procedure: passaggio dei parametri, convenzioni sui registri, stack frame, procedure annidate e ricorsive. Infine si fa il percorso inverso, dall'assembly al C. Alla fine del modulo si sanno scrivere e leggere programmi RISC-V con cicli, array e chiamate di funzione, e provarli nel simulatore.

> **Aggiunta in revisione · fonte: slide del corso (Gestione chiamate a funzione, Vettori e matrici, Fibonacci, Merge Sort, Reverse coding, Simulatore QtRvSim), programmi di esempio `Simple.s` e `Strlen.s`; Patterson e Hennessy, *Struttura e progetto dei calcolatori*, cap. 2 (ADC-02)**
>
> Il modulo è scritto in revisione dal materiale del corso. Tutti i frammenti di codice sono stati eseguiti con il simulatore di `strumenti/verifica_ampliamento.py`. Nelle slide alcuni esempi usano la notazione MIPS (`$t0`, `$s1`); qui si usa sempre quella RISC-V.

2.1 Dal C all'assembly: espressioni e condizioni

Un compilatore assegna le variabili a registri e traduce ogni espressione in una sequenza di istruzioni con risultati intermedi nei temporanei. Con `f`, `g`, `h`, `i`, `j` in `s0`…`s4`, l'istruzione C `f = (g + h) - (i + j);` diventa

```
add  t0, s1, s2      # t0 = g + h
add  t1, s3, s4      # t1 = i + j
sub  s0, t0, t1      # f = t0 - t1
```

Le condizioni diventano salti condizionati, di solito con la **condizione negata**: si salta oltre il ramo "vero" quando la condizione è falsa. Per `if (i == j) f = g + h; else f = g - h;`:

```
      bne  s3, s4, Else   # se i != j salta al ramo else
      add  s0, s1, s2     # f = g + h
      j    Fine           # salta oltre il ramo else
Else: sub  s0, s1, s2     # f = g - h
Fine:
```

Un ciclo è un salto all'indietro. `while (v[i] == k) i += 1;`, con `i` in `s3`, `k` in `s5` e l'indirizzo di `v` in `s6`:

```
Ciclo: slli t1, s3, 2     # t1 = 4 * i (ogni intero occupa 4 byte)
       add  t1, t1, s6    # t1 = indirizzo di v[i]
       lw   t0, 0(t1)     # t0 = v[i]
       bne  t0, s5, Esci  # se v[i] != k esci
       addi s3, s3, 1     # i += 1
       j    Ciclo
Esci:
```

Un `for (i = 0; i < n; i++)` si traduce allo stesso modo: inizializzazione prima del ciclo, controllo con `bge` (uscita se $i \geq n$) all'inizio, incremento e salto all'indietro alla fine.

2.2 Array e matrici

L'indirizzo dell'elemento `v[i]` di un array di interi è $\text{base} + 4i$: l'indice va moltiplicato per la dimensione dell'elemento, cioè shiftato a sinistra di 2 (`slli`). Per un array di `char` la dimensione è 1 e lo shift non serve. È la differenza principale fra l'indice del C, che conta elementi, e l'indirizzo, che conta byte.

Una matrice C `int M[R][C]` è memorizzata **per righe** (*row-major*): prima tutta la riga 0, poi la riga 1, e così via. L'elemento `M[i][j]` ha indirizzo

$$\text{base} + (i \cdot C + j) \cdot 4.$$

Se C è una potenza di 2 la moltiplicazione $i \cdot C$ è uno shift; altrimenti si usa `mul` (estensione M di RISC-V) o una somma ripetuta. Scorrendo la matrice con un puntatore che avanza di 4 byte si visitano gli elementi nell'ordine della memoria, riga per riga.

Somma degli elementi di un array

La funzione `int somma(int v[], int n)` riceve l'indirizzo di `v` in `a0` e `n` in `a1`, e restituisce il risultato in `a0`.

```
somma:  li   t0, 0          # i = 0
        li   t1, 0          # s = 0
ciclo:  bge  t0, a1, fine   # se i >= n esci
        slli t2, t0, 2      # t2 = 4 * i
        add  t2, a0, t2     # t2 = &v[i]
        lw   t3, 0(t2)      # t3 = v[i]
        add  t1, t1, t3     # s += v[i]
        addi t0, t0, 1      # i++
        j    ciclo
fine:   mv   a0, t1         # risultato in a0
        ret
```

2.3 Byte e stringhe

Una stringa C è un array di byte terminato dal carattere nullo (codice 0). Si legge un carattere alla volta con `lbu` (senza segno: i codici vanno da 0 a 255) e si scrive con `sb`. La lunghezza di una stringa si calcola facendo avanzare un puntatore fino allo 0:

```
strlen: mv   t0, a0         # t0 = puntatore al carattere corrente
loop:   lbu  t1, 0(t0)      # t1 = *t0
        beq  t1, zero, end  # fine della stringa
        addi t0, t0, 1      # carattere successivo
        j    loop
end:    sub  a0, t0, a0     # lunghezza = puntatore finale - iniziale
        ret
```

Qui si usa l'**aritmetica dei puntatori** invece di un indice: invece di calcolare a ogni giro $\text{base} + i$, si incrementa direttamente l'indirizzo, e si risparmiano istruzioni. È la stessa ottimizzazione che fa un compilatore.

2.4 Il modello di memoria di un programma

Lo spazio di indirizzamento di un programma RISC-V è diviso in zone, dal basso verso l'alto:

Zona	Contenuto
riservata	indirizzi bassi non usati
testo (*text*)	il codice macchina
dati statici (*data*)	variabili globali e costanti, raggiunte tramite `gp`
heap	memoria allocata dinamicamente (`malloc`, `new`), cresce verso l'alto
stack	variabili locali e dati delle chiamate, cresce verso il basso da `sp`

Heap e stack crescono uno verso l'altro, così lo spazio libero in mezzo può servire all'uno o all'altro. Nel sorgente assembly le zone si scelgono con le **direttive**: `.text` per il codice, `.data` per i dati; `.word 3, 5, 7` riserva e inizializza parole, `.asciz "ciao"` una stringa con lo 0 finale, `.space n` riserva n byte. Nel simulatore QtRvSim del corso il programma comincia all'etichetta `_start` (dichiarata con `.globl _start`), e si possono seguire passo per passo i registri, la memoria e la pipeline; il programma termina con una chiamata di sistema (`ecall`) o con un ciclo infinito di arresto.

2.5 Le procedure

Una chiamata di procedura richiede sei passi: (1) il chiamante mette gli argomenti dove la procedura li cerca; (2) trasferisce il controllo; (3) la procedura si procura la memoria che le serve; (4) svolge il suo compito; (5) mette il risultato dove il chiamante lo cerca; (6) restituisce il controllo al punto da cui era stata chiamata.

In RISC-V gli argomenti vanno in `a0`…`a7` (gli altri, se ce ne sono di più, sullo stack), il risultato in `a0` (e `a1`). La chiamata è `jal ra, procedura` (scritta anche `call procedura`), che salva in `ra` l'indirizzo di ritorno; il ritorno è `jalr zero, 0(ra)`, cioè `ret`.

Il problema è che chiamante e procedura usano gli stessi 32 registri. La **convenzione di chiamata** divide il lavoro:

- i registri **salvati** `s0`…`s11`, `sp` e `ra` (dal punto di vista del chiamante) devono avere, al ritorno, lo stesso valore che avevano alla chiamata: se la procedura li usa, li salva sullo stack all'inizio e li ripristina alla fine (*callee-saved*);
- i registri **temporanei** `t0`…`t6` e gli argomenti `a0`…`a7` possono essere modificati dalla procedura: se il chiamante ne ha bisogno dopo la chiamata, è lui a salvarli (*caller-saved*).

Lo **stack** si usa così: per riservare k byte si sottrae k a `sp` (`addi sp, sp, -k`), si salvano i registri con `sw` a offset positivi da `sp`, e alla fine si ripristinano e si rilascia lo spazio (`addi sp, sp, k`). La zona di stack di una chiamata si chiama **stack frame** (o record di attivazione). `sp` resta allineato a 16 byte nella convenzione standard; negli esempi del corso, per semplicità, si riservano solo i byte necessari.

Una procedura che non ne chiama altre (**procedura foglia**) spesso non ha bisogno dello stack: usa i temporanei e non tocca `ra`. Una procedura che ne chiama altre deve salvare almeno `ra`, perché la sua `jal` lo sovrascrive, e i registri che le servono dopo la chiamata.

2.6 Procedure ricorsive

Una procedura ricorsiva è una procedura non foglia che chiama sé stessa: ogni chiamata ha il suo stack frame con i suoi valori.

Fattoriale

```
int fact(int n) { return n < 2 ? 1 : n * fact(n - 1); }
```

```
fact:   addi sp, sp, -8     # spazio per ra e n
        sw   ra, 4(sp)
        sw   a0, 0(sp)      # salva n
        li   t0, 2
        blt  a0, t0, base   # n < 2: caso base
        addi a0, a0, -1
        jal  ra, fact       # a0 = fact(n - 1)
        lw   t1, 0(sp)      # t1 = n (salvato prima della chiamata)
        mul  a0, a0, t1     # a0 = n * fact(n - 1)
        j    esci
base:   li   a0, 1
esci:   lw   ra, 4(sp)
        addi sp, sp, 8
        ret
```

Per `fact(3)` lo stack contiene, al momento più profondo, tre frame con $n = 3, 2, 1$ e i rispettivi indirizzi di ritorno; i frame si liberano in ordine inverso mentre si calcolano 1, $2 \cdot 1$, $3 \cdot 2$.

Fibonacci

`int fib(int n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }`. Dopo la prima chiamata ricorsiva serve ancora sia n sia $fib(n-1)$: si tengono in due registri salvati, `s0` e `s1`, che vanno quindi salvati sullo stack.

```
fib:    li   t0, 2
        blt  a0, t0, fine   # n < 2: restituisce n, che è già in a0
        addi sp, sp, -12
        sw   ra, 8(sp)
        sw   s0, 4(sp)
        sw   s1, 0(sp)
        mv   s0, a0         # s0 = n
        addi a0, s0, -1
        jal  ra, fib        # a0 = fib(n - 1)
        mv   s1, a0         # s1 = fib(n - 1)
        addi a0, s0, -2
        jal  ra, fib        # a0 = fib(n - 2)
        add  a0, s1, a0
        lw   s1, 0(sp)
        lw   s0, 4(sp)
        lw   ra, 8(sp)
        addi sp, sp, 12
fine:   ret
```

Nel caso base la procedura è foglia e non tocca lo stack. Il numero di chiamate cresce come i numeri di Fibonacci stessi: `fib(10)` = 55 richiede 177 chiamate. La versione con un ciclo ne richiede una.

2.7 Dall’assembly al C

Leggere un programma assembly vuol dire ricostruire le variabili (a quali registri corrispondono), i cicli (i salti all’indietro) e le condizioni (i salti in avanti, con la condizione negata). Si procede così: si individuano gli argomenti (`a0`, `a1`…) e il valore di ritorno; si cerca il salto all’indietro, che chiude un ciclo, e il salto di uscita; si traduce il corpo istruzione per istruzione; infine si riscrive il tutto in un C leggibile, con i nomi delle variabili e un `for` dove c’è un contatore.

Esempi ed esercizi svolti

Aggiunta in revisione · fonte: esercizi nuovi sul modello delle slide (Vettori e matrici, Reverse coding, Fibonacci); tutti i programmi eseguiti con il simulatore di `strumenti/verifica_ampliamento.py` (ADC-02)

Esercizio 1. Il massimo di un array. Scrivere `int max(int v[], int n)` (con $n \geq 1$).

```
max:    lw   t1, 0(a0)      # m = v[0]
        li   t0, 1          # i = 1
ciclo:  bge  t0, a1, fine
        slli t2, t0, 2
        add  t2, a0, t2
        lw   t3, 0(t2)      # t3 = v[i]
        bge  t1, t3, avanti # se m >= v[i] non cambia
        mv   t1, t3         # m = v[i]
avanti: addi t0, t0, 1
        j    ciclo
fine:   mv   a0, t1
        ret
```

Con $v = (3, -2, 9, 4)$ restituisce 9. La condizione del C è `if (v[i] > m) m = v[i];`, e il salto la nega: si salta oltre l’assegnamento se $m \geq v[i]$.

Esercizio 2. La traccia di una matrice. `M` è una matrice $N \times N$ di interi, con l'indirizzo in `a0` e N in `a1`. Scrivere la somma degli elementi della diagonale.

L'elemento `M[i][i]` è all'indirizzo base $+ (iN + i) \cdot 4 =$ base $+ i(N+1) \cdot 4$: passando da un elemento della diagonale al successivo l'indirizzo cresce di $4(N+1)$. Conviene quindi un puntatore che avanza di quel passo:

```
traccia: addi t2, a1, 1     # t2 = N + 1
         slli t2, t2, 2     # passo = 4(N + 1) byte
         li   t0, 0         # i = 0
         li   t1, 0         # s = 0
ciclo:   bge  t0, a1, fine
         lw   t3, 0(a0)     # M[i][i]
         add  t1, t1, t3
         add  a0, a0, t2    # puntatore al prossimo elemento della diagonale
         addi t0, t0, 1
         j    ciclo
fine:    mv   a0, t1
         ret
```

Per la matrice 3×3 con elementi $1, \dots, 9$ per righe la traccia è $1 + 5 + 9 = 15$.

Esercizio 3. Reverse coding. Che cosa calcola questa funzione, con l'indirizzo di un array in `a0` e la sua lunghezza in `a1`?

```
        li   t0, 0
        li   t1, 0
L1:     bge  t0, a1, L3
        slli t2, t0, 2
        add  t2, a0, t2
        lw   t3, 0(t2)
        bge  t3, zero, L2
        addi t1, t1, 1
L2:     addi t0, t0, 1
        j    L1
L3:     mv   a0, t1
        ret
```

`t0` parte da 0, cresce di 1 e il ciclo esce quando arriva ad `a1`: è un indice i da 0 a $n-1$. Il corpo carica `v[i]` e, se **non** è ≥ 0, cioè se è negativo, incrementa `t1`, che è restituito. In C:

```c
int conta_negativi(int v[], int n) {
    int c = 0;
    for (int i = 0; i < n; i++)
        if (v[i] < 0) c++;
    return c;
}
```

Esercizio 4. Una procedura che ne chiama un'altra. Scrivere `int somma_quadrati(int a, int b)` che restituisce $a^2 + b^2$ usando una procedura foglia `quad(x)` che restituisce x^2.

```
quad:   mul  a0, a0, a0     # foglia: niente stack
        ret

somma_quadrati:
        addi sp, sp, -8
        sw   ra, 4(sp)      # la jal sotto sovrascrive ra
        sw   s0, 0(sp)      # s0 serve dopo la chiamata
        mv   s0, a1         # tiene b al sicuro
        jal  ra, quad       # a0 = a^2
        mv   t0, a0
```

```
        mv   a0, s0
        mv   s0, t0         # s0 = a^2
        jal  ra, quad       # a0 = b^2
        add  a0, a0, s0
        lw   s0, 0(sp)
        lw   ra, 4(sp)
        addi sp, sp, 8
        ret
```

`b` non può restare in `a1`, perché per la convenzione `quad` potrebbe cambiarlo; e a^2 non può restare in `t0` durante la seconda chiamata, per lo stesso motivo. Per questo entrambi passano da `s0`, salvato sullo stack. Con $a = 3$ e $b = 4$ restituisce 25.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; programmi eseguiti con il simulatore di `strumenti/verifica_ampliamento.py` (ADC-02)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. Tradurre un ciclo con una condizione. Tradurre `int s = 0; for (int i = 0; i < n; i++) if (v[i] % 2 == 0) s += v[i];` (somma dei pari), con `v` in `a0`, `n` in `a1`, risultato in `a0`.

Passo 1: i registri. `i` in `t0`, `s` in `t1`, temporanei `t2`, `t3`.

Passo 2: il test di parità senza divisione. Un numero è pari se il bit meno significativo è 0: `andi t4, t3, 1`.

Passo 3: la condizione negata. Il C dice "se è pari somma": si salta oltre la somma se è dispari, `bne t4, zero, avanti`.

Passo 4: il codice.

```
        li   t0, 0
        li   t1, 0
ciclo:  bge  t0, a1, fine
        slli t2, t0, 2
        add  t2, a0, t2
        lw   t3, 0(t2)
        andi t4, t3, 1
        bne  t4, zero, avanti
        add  t1, t1, t3
avanti: addi t0, t0, 1
        j    ciclo
fine:   mv   a0, t1
        ret
```

Passo 5: il controllo. Con $v = (1, 2, 3, 4, -6)$ la somma dei pari è $2 + 4 - 6 = 0$. Il test con `andi` funziona anche per i negativi in complemento a 2 (-6 = …`11111010`, bit 0 a 0).

Guidato 2. Disegnare lo stack. Durante `fact(3)` dell'esempio del paragrafo 2.6, con `sp` = `0x1000` all'inizio, che cosa c'è nello stack quando si raggiunge il caso base?

Passo 1: la prima chiamata. `sp` scende a `0x0FF8`; in `0x0FFC` c'è `ra` del chiamante esterno, in `0x0FF8` c'è $n = 3$.

Passo 2: la seconda. `sp` = `0x0FF0`; in `0x0FF4` l'indirizzo di ritorno dentro `fact` (l'istruzione dopo la `jal`), in `0x0FF0` $n = 2$.

Passo 3: la terza, caso base. `sp` = `0x0FE8`; in `0x0FEC` lo stesso indirizzo di ritorno, in `0x0FE8` $n = 1$. Con $n = 1 < 2$ si va al caso base.

Passo 4: la risalita. Ogni ritorno rilascia 8 byte: `sp` torna a `0x0FF0`, `0x0FF8`, `0x1000`, mentre `a0` vale 1, 2, 6.

Guidato 3. Reverse coding con un puntatore. Che cosa fa questa funzione, con `a0` e `a1` indirizzi di due stringhe?

```
f:      lbu  t0, 0(a1)
        sb   t0, 0(a0)
        beq  t0, zero, fine
        addi a0, a0, 1
        addi a1, a1, 1
        j    f
fine:   ret
```

Passo 1: che cosa si legge e si scrive. Si legge un byte da `a1` e lo si scrive in `a0`.

Passo 2: il ciclo. Si avanza di un byte su entrambe le stringhe finché il byte copiato non è lo 0.

Passo 3: il C. `void copia(char *dst, const char *src) { while ((*dst = *src) != 0) { dst++; src++; } }`: è `strcpy`. Lo 0 finale viene copiato prima del test, così la stringa di destinazione è terminata correttamente.

Riepilogo

Il compilatore assegna le variabili ai registri e traduce condizioni e cicli in salti condizionati con la condizione negata. L'indirizzo di `v[i]` è base $+ 4i$ per gli interi, e una matrice è memorizzata per righe, con `M[i][j]` a base $+ (iC + j) \cdot 4$; scorrere con un puntatore invece che con un indice risparmia istruzioni. La memoria di un programma è divisa in testo, dati statici, heap e stack; le direttive dell'assemblatore scelgono la zona. Nelle procedure gli argomenti passano in `a0`…`a7`, il risultato in `a0`, la chiamata è `jal ra` e il ritorno `jalr zero, 0(ra)`; i registri `s` e `ra` sono preservati da chi li usa, i `t` e gli `a` da chi ne ha bisogno dopo la chiamata. Una procedura non foglia salva `ra` e i registri salvati nel suo stack frame; la ricorsione funziona perché ogni chiamata ha il suo frame.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (ADC-02)**

Schemi di traduzione

C	RISC-V
`if (a == b) X; else Y;`	`bne a, b, Else` · X · `j Fine` · `Else:` Y · `Fine:`
`while (cond) X;`	`Ciclo:` salto fuori se non cond · X · `j Ciclo`
`for (i = 0; i < n; i++) X;`	`li i, 0` · `Ciclo: bge i, n, Fine` · X · `addi i, i, 1` · `j Ciclo`
`v[i]` (int)	`slli t, i, 2` · `add t, base, t` · `lw x, 0(t)`
`M[i][j]` (int, C colonne)	indirizzo base $+ (iC + j) \cdot 4$
`c = s[i]` (char)	`add t, base, i` · `lbu c, 0(t)`

Convenzione di chiamata

Chi salva	Registri
la procedura chiamata (callee-saved)	`s0`–`s11`, `sp`, `ra` (se ne chiama altre)
il chiamante (caller-saved)	`t0`–`t6`, `a0`–`a7`

Prologo: `addi sp, sp, -k` · `sw ra, …(sp)` · `sw s…, …(sp)`. Epilogo: `lw` in ordine inverso · `addi sp, sp, k` · `ret`.

Termini per l'orale

Termine	In una riga
Procedura foglia	non chiama altre procedure, spesso non usa lo stack

Termine	In una riga
Stack frame	zona di stack di una chiamata: registri salvati e variabili locali
Indirizzo di ritorno	indirizzo dell'istruzione dopo la chiamata, salvato in `ra`
Caller-saved / callee-saved	salvati da chi chiama / da chi è chiamato
Row-major	memorizzazione delle matrici riga per riga
Aritmetica dei puntatori	avanzare un indirizzo invece di ricalcolarlo da un indice
Direttiva	comando per l'assemblatore (`.data`, `.word`, `.asciz`)

Domande che tornano spesso

- Quali sono i passi di una chiamata di procedura e come li realizza RISC-V?
- Perché alcuni registri sono preservati e altri no? Che cosa deve salvare una procedura non foglia?
- Disegnare lo stack durante una chiamata ricorsiva.
- Come si calcola l'indirizzo di un elemento di un array e di una matrice?
- Descrivere le zone della memoria di un programma.
- Tradurre in C un frammento assembly assegnato.

3 · L’ALU e l’aritmetica del calcolatore

In questo modulo. Si usano il sommatore e il complemento a 2 delle reti logiche e le istruzioni del modulo 1 (*Astrazione, prestazioni e l’ISA RISC-V*). Si costruisce passo per passo l’unità aritmetico-logica che esegue le istruzioni RISC-V: prima un bit, poi 32, con la sottrazione, il confronto `slt`, il test di zero e il rilevamento dell’overflow. Poi si vede come rendere veloce la somma con il calcolo anticipato del riporto, e come si fanno in hardware la moltiplicazione (anche con l’algoritmo di Booth) e la divisione. La terza parte è la virgola mobile IEEE 754, con gli algoritmi di somma e prodotto. Chiudono i codici di Hamming, che proteggono i dati in memoria dagli errori. Alla fine del modulo si sanno eseguire a mano tutti gli algoritmi e spiegare l’hardware che li realizza.

> **Aggiunta in revisione · fonte: slide del corso (Arithmetic Logic Unit, Codici di Hamming); Patterson e Hennessy, *Struttura e progetto dei calcolatori*, edizione RISC-V, cap. 3 e appendice A (ADC-03)**
>
> Il modulo è scritto in revisione dal materiale del corso; ogni algoritmo è svolto su un esempio controllato con `strumenti/verifica_ampliamento.py`.

3.1 L’ALU a un bit

L’ALU deve eseguire le operazioni dell’ISA sugli interi: `and`, `or`, somma, sottrazione, confronto. Si comincia da un solo bit. Con gli ingressi a e b si calcolano in parallelo $a \cdot b$, $a + b$ (OR) e la somma con un full adder, che ha anche il riporto in ingresso c_{in} e quello in uscita c_{out}. Un multiplexer comandato dal segnale **Operation** sceglie quale risultato portare in uscita.

La sottrazione non richiede un circuito nuovo: in complemento a 2, $a - b = a + \bar{b} + 1$. Si aggiunge un multiplexer che sceglie fra b e $\bar{b}$ (segnale **Binvert**), e per il “+1” si mette a 1 il riporto in ingresso del bit 0. Con un secondo multiplexer su a (**Ainvert**) e le leggi di De Morgan si ottiene anche la NOR: $\overline{a + b} = \bar{a} \cdot \bar{b}$.

3.2 L’ALU a 32 bit

Mettendo in fila 32 ALU a un bit, con il c_{out} di ognuna collegato al c_{in} della successiva, si ottiene un sommatore a propagazione del riporto (*ripple carry*) che sa anche fare AND, OR, NOR e sottrazione. Servono tre aggiunte.

`slt`. Il risultato è 1 se $a < b$ e 0 altrimenti, cioè un numero con tutti i bit a 0 tranne forse il bit 0. Si calcola $a - b$: se il risultato è negativo, $a < b$. Ogni ALU a un bit ha un ingresso in più, **Less**, selezionato come quarta scelta del multiplexer: per i bit da 1 a 31 Less vale 0; per il bit 0 Less è collegato al bit di segno della sottrazione calcolato dall’ALU del bit 31 (un’uscita apposita, **Set**). Attenzione: se la sottrazione va in overflow il bit di segno è sbagliato, e il valore giusto è $\text{Set} \oplus \text{Overflow}$.

Zero. Per i salti `beq` e `bne` serve sapere se $a - b = 0$: è la NOR di tutti i 32 bit del risultato, $\text{Zero} = \overline{r_{31} + \cdots + r_0}$.

Overflow. In complemento a 2 c’è overflow quando il riporto che entra nel bit di segno è diverso da quello che ne esce, $V = c_{31} \oplus c_{32}$: succede solo sommando due numeri dello stesso segno (o sottraendo numeri di segno opposto). RISC-V non segnala l’overflow con un’eccezione: se serve, il programma lo controlla con un confronto.

Il controllo dell’ALU usa quattro bit: Ainvert, Binvert (che comanda anche il riporto in ingresso del bit 0) e i due bit di Operation.

Controllo ALU	Funzione
0000	AND
0001	OR
0010	somma
0110	sottrazione
0111	`slt`
1100	NOR

3.3 Il calcolo anticipato del riporto

Nel sommatore ripple il riporto attraversa tutti gli stadi, e il ritardo cresce con il numero di bit. Per rendere la somma veloce bisogna calcolare i riporti senza aspettare. Dalla tabella del full adder, $c_{i+1} = a_i b_i + (a_i + b_i) c_i$. Si definiscono per ogni bit

$$g_i = a_i b_i \quad \text{(generazione)}, \qquad p_i = a_i + b_i \quad \text{(propagazione)},$$

perché lo stadio i *genera* un riporto se $a_i = b_i = 1$, qualunque sia c_i, e *propaga* il riporto in arrivo se almeno uno dei due bit vale 1. Allora $c_{i+1} = g_i + p_i c_i$, e sostituendo all'indietro:

$$c_1 = g_0 + p_0 c_0, \qquad c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0,$$

$$c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0, \qquad c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0.$$

Ogni riporto è una rete a due livelli in g, p e c_0: tutti i riporti sono pronti dopo lo stesso tempo (**carry lookahead**). Il prezzo è che le espressioni crescono rapidamente; per 32 bit si raggruppano i bit a blocchi di 4, ciascuno con il suo G e P di blocco ($P_0 = p_3 p_2 p_1 p_0$, $G_0 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0$), e si applica lo stesso schema ai blocchi.

3.4 La moltiplicazione

Il prodotto di due numeri di n bit ha fino a $2n$ bit. L'algoritmo è quello in colonna: per ogni bit a 1 del moltiplicatore si somma il moltiplicando spostato. L'hardware delle slide del corso (*add and right shift*) usa un registro prodotto di $2n$ bit: all'inizio la metà alta vale 0 e la metà bassa contiene il moltiplicatore. A ogni passo, se il bit meno significativo del registro è 1 si somma il moltiplicando alla metà alta; poi si sposta tutto il registro di un bit a destra. Dopo n passi il registro contiene il prodotto. Serve un solo sommatore di n bit.

Per i numeri con segno si può moltiplicare i valori assoluti e sistemare il segno alla fine, oppure usare l'**algoritmo di Booth**. L'idea è che una sequenza di 1 nel moltiplicatore si può sostituire con una sottrazione e una somma: $0110 = 1000 - 0010$, quindi $m \times 0110 = m \times 1000 - m \times 0010$. Si guarda la coppia formata dal bit corrente del moltiplicatore e dal bit alla sua destra (all'inizio un bit aggiuntivo $q_{-1} = 0$):

$q_0\ q_{-1}$	**Operazione**	**Significato**
1 0	$A = A - M$	inizio di una sequenza di 1
1 1	nessuna	in mezzo alla sequenza
0 1	$A = A + M$	fine della sequenza
0 0	nessuna	in mezzo a una sequenza di 0

e dopo ogni operazione si fa uno shift **aritmetico** a destra (che conserva il segno) di A, Q e q_{-1} insieme. Funziona direttamente in complemento a 2.

In RISC-V la moltiplicazione è nell'estensione M: `mul rd, rs1, rs2` dà i 32 bit bassi del prodotto, `mulh` (con segno) e `mulhu` (senza segno) quelli alti; per sapere se un prodotto sta in 32 bit si guarda la parte alta.

3.5 La divisione

La divisione si fa con sottrazioni e shift, come quella in colonna: $\text{dividendo} = \text{quoziente} \times \text{divisore} + \text{resto}$, con $0 \le \text{resto} < \text{divisore}$. A ogni passo si prova a sottrarre il divisore (allineato) dal resto parziale: se il risultato è ≥ 0 il bit del quoziente è 1 e si tiene la sottrazione; se è negativo il bit è 0 e si ripristina il resto (algoritmo *restoring*). In hardware il divisore sta nella metà alta di un registro di $2n$ bit e scorre a destra di un bit a ogni passo, mentre il quoziente si forma a sinistra. A differenza della moltiplicazione, i passi non si possono fare in parallelo, perché ognuno dipende dal segno del precedente: per questo la divisione è l'operazione intera più lenta. In RISC-V: `div`, `divu`, `rem`, `remu`; la divisione per zero non genera eccezioni ma dà un risultato convenzionale (quoziente con tutti i bit a 1).

3.6 La virgola mobile

Il formato IEEE 754 è spiegato nel modulo 2 di Reti Logiche; qui si ricorda l'essenziale e si vedono le operazioni. Un numero normalizzato in singola precisione è

$$(-1)^s \times (1 + F) \times 2^{E-127},$$

con 1 bit di segno, 8 di esponente in eccesso 127 e 23 di frazione F (il bit 1 prima della virgola è nascosto). In doppia precisione l'esponente ha 11 bit (eccesso 1023) e la frazione 52. Si ha **overflow** quando l'esponente supera il massimo rappresentabile, **underflow** quando un numero diverso da 0 è più piccolo del minimo normalizzato; i valori con esponente tutto a 0 e tutto a 1 sono riservati a zero, denormalizzati, infiniti e NaN.

Somma. (1) Si confrontano gli esponenti e si sposta a destra la mantissa del numero con l'esponente minore, finché gli esponenti sono uguali. (2) Si sommano le mantisse (con il segno). (3) Si normalizza il risultato, spostando la mantissa e correggendo l'esponente, e si controlla overflow o underflow. (4) Si arrotonda ai bit disponibili; se l'arrotondamento denormalizza il risultato, si torna al passo 3.

Prodotto. (1) Si sommano gli esponenti *veri*: con gli esponenti in eccesso bisogna togliere un 127, perché $(E_1 + 127) + (E_2 + 127) = (E_1 + E_2 + 127) + 127$. (2) Si moltiplicano le mantisse. (3) Si normalizza e si controlla overflow e underflow. (4) Si arrotonda. (5) Il segno è lo XOR dei segni.

Per arrotondare bene l'hardware tiene, durante i calcoli, due bit in più a destra (**guard** e **round**) e un bit **sticky** che vale 1 se qualcuno dei bit scartati era 1. Il modo di arrotondamento standard è al più vicino, e in caso di pareggio al pari.

3.7 Rilevare e correggere gli errori: i codici di Hamming

Un bit in memoria o su un canale può cambiare per un disturbo. La **distanza di Hamming** fra due sequenze di bit della stessa lunghezza è il numero di posizioni in cui differiscono: $d(1011, 1110) = 2$. La distanza di un codice è la minima distanza fra due parole valide. Con distanza d si possono rilevare fino a $d - 1$ errori, oppure correggerne fino a $\left\lfloor \frac{d-1}{2} \right\rfloor$: con $d = 2$ (il bit di parità) si rileva un errore singolo, con $d = 3$ lo si corregge, con $d = 4$ si corregge un errore e se ne rilevano due.

Per correggere un errore su M bit di dati si aggiungono K bit di controllo, che insieme devono indicare se non c'è errore o in quale delle $M + K$ posizioni è avvenuto. Servono quindi

$$2^K \geq M + K + 1.$$

Per $M = 4$ basta $K = 3$ (codice di Hamming (7, 4)); per $M = 8$, $K = 4$; per $M = 16$, $K = 5$; per $M = 32$, $K = 6$; per $M = 64$, $K = 7$. La frazione di bit di controllo diminuisce al crescere di M.

Costruzione. Si numerano le posizioni da 1: i bit di controllo vanno nelle posizioni potenze di 2 (1, 2, 4, 8, …), i dati nelle altre. Il bit di controllo in posizione 2^j è la parità (pari) delle posizioni il cui numero, in binario, ha il bit j a 1: il bit 1 controlla le posizioni 1, 3, 5, 7, …; il bit 2 le posizioni 2, 3, 6, 7, …; il bit 4 le posizioni 4, 5, 6, 7, … Chi riceve ricalcola le tre parità: i risultati, letti come numero binario (la **sindrome**), valgono 0 se non c'è errore, e altrimenti danno proprio la posizione del bit sbagliato, che basta invertire. Aggiungendo un bit di parità su tutta la parola si ottiene il codice **SECDED** (correzione di un errore, rilevazione di due), usato nelle memorie ECC.

Esempi ed esercizi svolti

> **Aggiunta in revisione · fonte: esercizi nuovi sul modello delle slide (ALU, Codici di Hamming); soluzioni controllate con `strumenti/verifica_ampliamento.py` (ADC-03)**

Esercizio 1. I riporti anticipati di una somma a 4 bit. $a = 1011$, $b = 0110$, $c_0 = 0$.

$g = a \cdot b$ bit per bit $= 0010$, $p = a + b = 1111$. Quindi $c_1 = g_0 = 0$; $c_2 = g_1 + p_1 g_0 = 1$; $c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 = 0 + 1 + 0 = 1$; $c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + \cdots = 0 + 0 + 1 = 1$. Le somme sono $s_i = a_i \oplus b_i \oplus c_i$: $s_0 = 1$, $s_1 = 0$, $s_2 = 0$, $s_3 = 0$, e il risultato è $1\,0001$, cioè $11 + 6 = 17$. Il riporto finale nasce in g_1 e viaggia attraverso p_2 e p_3: nel ripple impiegherebbe tre stadi, con il lookahead è pronto subito.

Esercizio 2. Moltiplicazione con l'hardware add and right shift. 6×5 con 4 bit: moltiplicando $M = 0110$, registro prodotto iniziale $0000\,0101$.

Passo	Bit basso	Dopo l'eventuale somma	Dopo lo shift
1	1	0110 0101	0011 0010
2	0	–	0001 1001
3	1	0111 1001	0011 1100
4	0	–	0001 1110

Il risultato è $0001\,1110 = 30$. A ogni passo il moltiplicatore "esce" a destra e il prodotto parziale "entra" da sinistra: lo stesso registro contiene entrambi.

Esercizio 3. Booth con un moltiplicatore negativo. $3 \times (-2)$ con 4 bit: $M = 0011$, $Q = 1110$, $q_{-1} = 0$, $A = 0000$.

Passo	$q_0\, q_{-1}$	Operazione	A dopo lo shift	Q dopo lo shift	q_{-1}
1	0 0	nessuna	0000	0111	0
2	1 0	$A = A - M = 1101$	1110	1011	1
3	1 1	nessuna	1111	0101	1
4	1 1	nessuna	1111	1010	1

Il prodotto è $A\,Q = 1111\,1010 = -6$ in complemento a 2. Il moltiplicatore 1110 è una sola sequenza di 1 che arriva fino al bit di segno: c'è una sola sottrazione e nessuna somma, perché la sequenza non "finisce".

Esercizio 4. Divisione in binario. $1011 \div 0011$ ($11 \div 3$).

Si procede come in colonna, abbassando un bit alla volta: $1 < 11$, quoziente 0; $10 < 11$, quoziente 0; $101 \geq 11$, quoziente 1 e resto $101 - 11 = 10$; si abbassa l'ultimo bit, $101 \geq 11$, quoziente 1 e resto 10. Quoziente $0011 = 3$, resto $10 = 2$, e infatti $3 \cdot 3 + 2 = 11$.

Esercizio 5. Somma in virgola mobile. $1{,}75 + 0{,}3125$.

$1{,}75 = 1{,}11_2 \times 2^0$ e $0{,}3125 = 1{,}01_2 \times 2^{-2}$. (1) Si allinea il secondo all'esponente 0: $0{,}0101_2 \times 2^0$. (2) Somma: $1{,}1100 + 0{,}0101 = 10{,}0001_2$. (3) Normalizzazione: $1{,}00001_2 \times 2^1$. (4) Con 23 bit di frazione non serve arrotondare. Il risultato è $2 + \frac{2}{32} = 2{,}0625$.

Esercizio 6. Prodotto in virgola mobile. $1{,}5 \times (-2{,}25)$ in singola precisione.

$1{,}5 = 1{,}1_2 \times 2^0$ (campo esponente 127), $-2{,}25 = -1{,}001_2 \times 2^1$ (campo 128). (1) Esponente: $127 + 128 - 127 = 128$, cioè $E = 1$. (2) Mantisse: $1{,}1 \times 1{,}001 = 1{,}1011_2$. (3) Già normalizzata. (5) Segno: $0 \oplus 1 = 1$. Il risultato è $-1{,}1011_2 \times 2^1 = -3{,}375$.

Esercizio 7. Un codice di Hamming (7, 4). Codificare i dati 1011 e correggere la parola ricevuta se il bit in posizione 6 si altera.

I dati vanno nelle posizioni 3, 5, 6, 7: $b_3 = 1$, $b_5 = 0$, $b_6 = 1$, $b_7 = 1$. Parità: $b_1 = b_3 \oplus b_5 \oplus b_7 = 0$, $b_2 = b_3 \oplus b_6 \oplus b_7 = 1$, $b_4 = b_5 \oplus b_6 \oplus b_7 = 0$. La parola trasmessa, dalla posizione 1 alla 7, è $0\,1\,1\,0\,0\,1\,1$. Se si altera b_6 si riceve $0\,1\,1\,0\,0\,0\,1$. Il ricevitore calcola $s_1 = b_1 \oplus b_3 \oplus b_5 \oplus b_7 = 0$, $s_2 = b_2 \oplus b_3 \oplus b_6 \oplus b_7 = 1$, $s_4 = b_4 \oplus b_5 \oplus b_6 \oplus b_7 = 1$: la sindrome $s_4 s_2 s_1 = 110 = 6$ indica la posizione sbagliata, e invertendo b_6 si ritrova la parola giusta.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; soluzioni controllate con `strumenti/verifica_ampliamento.py` (ADC-03)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. L'overflow dell'ALU. Con 8 bit, l'ALU calcola $100 + 50$ e $(-100) - 50$. In quali casi segnala overflow e che cosa dà `slt` per $a = 100$, $b = -100$?

Passo 1. $100 + 50 = 150 > 127$: il risultato $1001\,0110$ ha segno negativo; i riporti nel e dal bit di segno sono $c_7 = 1$, $c_8 = 0$, diversi: overflow.

Passo 2. $(-100) - 50 = -150 < -128$: overflow anche qui, con risultato $0110\,1010$ apparentemente positivo.

Passo 3: `slt` di 100 *e* -100. $100 - (-100) = 200$ va in overflow, e il bit di segno della sottrazione è 1, come se $100 < -100$. Il valore giusto di Set è il bit di segno XOR overflow: $1 \oplus 1 = 0$, quindi `slt` dà 0, correttamente.

Guidato 2. Quanti bit di controllo? Si vuole correggere un errore singolo su parole di 32 bit, e rilevarne due.

Passo 1: la disuguaglianza. $2^K \geq 32 + K + 1$.

Passo 2: provare. $K = 5$: $32 \geq 38$, no. $K = 6$: $64 \geq 39$, sì.

Passo 3: SECDED. Per rilevare anche gli errori doppi si aggiunge un bit di parità globale: 7 bit di controllo su 32 di dati, una parola di 39 bit.

Passo 4: il confronto. Per 64 bit di dati bastano $K = 7$ più 1, cioè 8: è il formato delle memorie ECC, con 72 bit per ogni 64.

Guidato 3. Somma in virgola mobile con numeri di segno diverso. $0{,}5 + (-0{,}4375)$ con 4 bit di mantissa.

Passo 1: i numeri. $0{,}5 = 1{,}000_2 \times 2^{-1}$; $-0{,}4375 = -1{,}110_2 \times 2^{-2}$.

Passo 2: allineare. Il secondo diventa $-0{,}111_2 \times 2^{-1}$.

Passo 3: sommare. $1{,}000 - 0{,}111 = 0{,}001_2$, con esponente -1.

Passo 4: normalizzare. $0{,}001_2 \times 2^{-1} = 1{,}000_2 \times 2^{-4}$: lo shift è a sinistra di tre posti, e l'esponente scende di 3.

Passo 5: risultato. $2^{-4} = 0{,}0625$. La differenza di due numeri vicini perde cifre significative: è la **cancellazione**, il principale motivo di errore nei calcoli in virgola mobile.

Riepilogo

L'ALU a un bit sceglie con un multiplexer fra AND, OR e somma; con Binvert e il riporto iniziale a 1 sottrae, con Ainvert fa la NOR. L'ALU a 32 bit aggiunge l'ingresso Less per `slt`, l'uscita Zero per i salti e il rilevamento dell'overflow con $c_{31} \oplus c_{32}$. Il calcolo anticipato del riporto rende i riporti reti a due livelli nei segnali di generazione e propagazione, e si applica a blocchi. La moltiplicazione si fa con somme e shift su un registro di $2n$ bit; l'algoritmo di Booth sostituisce le sequenze di 1 con una sottrazione e una somma e lavora in complemento a 2. La divisione si fa con sottrazioni di prova e ripristini, un passo alla volta. In virgola mobile la somma richiede allineamento, somma, normalizzazione e arrotondamento; il prodotto somma gli esponenti togliendo l'eccesso e moltiplica le mantisse. I codici di Hamming con distanza 3 correggono un errore con K bit di controllo tali che $2^K \geq M + K + 1$, e la sindrome indica la posizione dell'errore.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (ADC-03)**

Interi

$$a - b = a + \bar{b} + 1, \qquad V = c_{n-1} \oplus c_n, \qquad \text{Zero} = \overline{r_{31} + \cdots + r_0}, \qquad \text{Set}_{slt} = s_{31} \oplus V.$$

$$g_i = a_i b_i, \quad p_i = a_i + b_i, \quad c_{i+1} = g_i + p_i c_i, \quad P = p_3 p_2 p_1 p_0, \quad G = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0.$$

Algoritmo	Regola del passo
Add and right shift	se il bit basso è 1, somma M alla metà alta; poi shift a destra
Booth	10: $A - M$; 01: $A + M$; 00, 11: niente; poi shift aritmetico a destra
Divisione restoring	sottrai; se negativo ripristina e il bit è 0, altrimenti il bit è 1

Virgola mobile

$$N = (-1)^s(1+F)2^{E-bias}, \quad bias = 127\ (32\text{ bit}), 1023\ (64\text{ bit}); \qquad E_{prodotto} = E_1 + E_2 - bias.$$

Somma: allinea, somma, normalizza, arrotonda. Prodotto: esponenti, mantisse, normalizza, arrotonda, segno. Bit di arrotondamento: guard, round, sticky.

Hamming

d = distanza minima: rileva $d-1$ errori, ne corregge $\lfloor (d-1)/2 \rfloor$. Bit di controllo: $2^K \geq M + K + 1$ (SECDED: $K+1$). Bit di controllo nelle posizioni $1, 2, 4, 8, \ldots$; il bit 2^j è la parità delle posizioni con il bit j a 1; la sindrome dà la posizione dell'errore.

Termini per l'orale

Termine	In una riga
Ripple carry	riporto che si propaga stadio per stadio
Carry lookahead	riporti calcolati in anticipo con generazione e propagazione
Algoritmo di Booth	moltiplicazione con somme e sottrazioni secondo le coppie di bit del moltiplicatore
Shift aritmetico	shift a destra che ripete il bit di segno
Normalizzazione	portare la mantissa nella forma $1,F$
Guard, round, sticky	bit extra per arrotondare correttamente
Distanza di Hamming	numero di bit in cui differiscono due parole
Sindrome	risultato dei controlli di parità, posizione dell'errore
SECDED	corregge un errore singolo e rileva quelli doppi

Domande che tornano spesso

- Costruire l'ALU a un bit e poi a 32 bit; come si realizzano sottrazione, `slt`, Zero e overflow?
- Perché il ripple carry è lento? Ricavare le equazioni del carry lookahead.
- Descrivere l'hardware della moltiplicazione e l'algoritmo di Booth, con un esempio.
- Perché la divisione è più lenta della moltiplicazione?
- Algoritmi di somma e prodotto in virgola mobile; perché nel prodotto si toglie il bias?
- Codici di Hamming: distanza, numero di bit di controllo, sindrome; che cos'è il SECDED?

4 · Il processore, la pipeline e il parallelismo fra istruzioni

In questo modulo. Si usano le istruzioni e i formati del modulo 1, l'ALU del modulo 3 e le reti sequenziali di Reti Logiche (registri, multiplexer, memorie). Si costruisce prima il percorso dei dati (*datapath*) che esegue in un solo ciclo di clock le istruzioni `add`, `sub`, `and`, `or`, `lw`, `sw` e `beq`, con l'unità di controllo che lo comanda. Poi si vede perché il ciclo singolo è lento e come la pipeline a cinque stadi aumenti il numero di istruzioni completate per unità di tempo. La parte più lunga riguarda i conflitti (*hazard*) della pipeline: strutturali, sui dati e sul controllo, con gli stalli, il forwarding, l'unità di rilevamento dei conflitti, la previsione dei salti e le eccezioni. Chiudono le architetture che avviano più istruzioni per ciclo: VLIW, srotolamento dei cicli, superscalari ed esecuzione fuori ordine. Alla fine del modulo si sa disegnare il diagramma temporale di un frammento di codice, contare gli stalli con e senza forwarding, riordinare il codice per eliminarli e calcolare il CPI di un programma.

> **Aggiunta in revisione · fonte: slide del corso (RISC-V Pipelined Architecture 1 e 2, Control Hazards, Multiple Issue), appunto a mano *Datapath con pipeline* dell'autore; Patterson e Hennessy, *Struttura e progetto dei calcolatori*, edizione RISC-V, cap. 4 (ADC-04, ADC-06, ADC-07)**
>
> Il modulo è scritto in revisione dal materiale del corso; l'appunto a mano dell'autore (lo schema del datapath con l'unità di forwarding e l'unità di rilevamento dei conflitti) è descritto alla fine di 4.7. Le slide usano in parte la notazione MIPS (i registri con il dollaro, il campo `Rt`): qui si usa quella RISC-V (`x1`, `rs1`, `rs2`, `rd`). I conteggi dei cicli sono controllati con `strumenti/pipe.py`.

4.1 Il datapath a ciclo singolo

Ogni istruzione RISC-V comincia allo stesso modo: si legge l'istruzione dalla memoria all'indirizzo contenuto nel PC (*program counter*) e si leggono uno o due registri, con i campi `rs1` e `rs2` che stanno sempre nella stessa posizione in tutti i formati. Poi le classi di istruzioni si separano, ma tutte usano l'ALU: le aritmetico-logiche per il calcolo, `lw` e `sw` per l'indirizzo, `beq` per il confronto. Infine `lw` e `sw` accedono alla memoria dati, le aritmetico-logiche e `lw` scrivono un registro, e il PC passa all'istruzione successiva ($PC + 4$) o alla destinazione del salto.

Gli elementi del datapath sono quindi il PC, la memoria delle istruzioni, il register file (due porte di lettura e una di scrittura), l'unità che estende in segno l'immediato, l'ALU, la memoria dei dati, un sommatore per $PC + 4$ e uno per l'indirizzo del salto ($PC + \text{immediato} \times 2$, perché nei salti l'immediato conta mezze parole). Dove un ingresso può arrivare da due sorgenti ci vuole un multiplexer: il secondo operando dell'ALU (registro o immediato, segnale **ALUSrc**), il dato da scrivere nel registro (uscita dell'ALU o della memoria, **MemtoReg**), il nuovo PC ($PC + 4$ o destinazione del salto, **PCSrc**). Gli elementi di stato (PC, registri, memorie) si aggiornano sul fronte del clock; tutto il resto è logica combinatoria.

Nel ciclo singolo ogni istruzione dura un ciclo, che deve bastare all'istruzione più lenta. Con i tempi delle slide (memorie e ALU 200 ps, register file 100 ps, il resto trascurabile):

Istruzione	Mem. istr.	Lettura reg.	ALU	Mem. dati	Scrittura reg.	Totale
tipo R	200	100	200	–	100	600 ps
`lw`	200	100	200	200	100	800 ps
`sw`	200	100	200	200	–	700 ps
`beq`	200	100	200	–	–	500 ps

Il clock deve durare 800 ps, quelli di `lw`, anche quando si esegue una `beq` che ne richiederebbe 500. È semplice da capire, ma usa male il tempo e anche l'area: un'unità funzionale si può usare una sola volta per ciclo, quindi servono due memorie separate e sommatori in più oltre all'ALU.

4.2 L'unità di controllo

L'unità di controllo legge il campo `opcode` dell'istruzione e genera i segnali che comandano i multiplexer, le scritture e l'ALU. Il controllo dell'ALU è a due livelli: l'unità principale produce un codice di due bit, **ALUOp**, e una piccola rete combina ALUOp con i campi `funct7` e `funct3` per ottenere i quattro bit di controllo del modulo

3. ALUOp = 00 vuol dire somma (per `lw` e `sw`, che calcolano l'indirizzo), 01 sottrazione (per `beq`, che confronta), 10 "guarda i campi funct" (per le istruzioni di tipo R).

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
tipo R	0	0	1	0	0	0	10
`lw`	1	1	1	1	0	0	00
`sw`	1	X	0	0	1	0	00
`beq`	0	X	0	0	0	1	01

La X (indifferenza) di MemtoReg in `sw` e `beq` è possibile perché RegWrite vale 0: qualunque cosa arrivi al register file, non viene scritta. Il segnale PCSrc è l'AND di Branch e dell'uscita Zero dell'ALU: si salta solo se l'istruzione è un salto *e* i due registri sono uguali. Con le tabelle di verità di questi segnali l'unità di controllo è una rete combinatoria, che si sintetizza come nel corso di Reti Logiche; nei processori più complessi la si realizza anche come macchina a stati o con un microprogramma.

4.3 La pipeline

L'idea è quella della catena di montaggio: si comincia a eseguire l'istruzione successiva prima che la precedente sia finita. Si divide l'esecuzione in cinque **stadi**, uno per ogni risorsa principale:

1. **IF** (*instruction fetch*): lettura dell'istruzione e aggiornamento del PC;
2. **ID** (*instruction decode*): decodifica e lettura dei registri;
3. **EX** (*execute*): operazione dell'ALU o calcolo dell'indirizzo;
4. **MEM**: accesso alla memoria dati;
5. **WB** (*write back*): scrittura del risultato nel register file.

Fra uno stadio e l'altro si mettono i **registri di pipeline** (IF/ID, ID/EX, EX/MEM, MEM/WB), che conservano tutto ciò che serve agli stadi successivi, compresi i segnali di controllo: l'unità di controllo li genera in ID e li fa "viaggiare" con l'istruzione, e ogni stadio usa i suoi. Il ciclo di clock è fissato dallo stadio più lento, 200 ps nell'esempio, invece degli 800 del ciclo singolo.

La pipeline non accorcia la **latenza** di una singola istruzione (anzi, la allunga: 5 × 200 = 1000 ps contro 800), ma aumenta il **throughput**: a regime si completa un'istruzione per ciclo. Con k stadi, ciclo T_c e n istruzioni senza conflitti,

$$T_{pipeline} = (k + n - 1)\,T_c,$$

perché la prima istruzione impiega k cicli e le altre ne escono una per ciclo. Per n grande l'accelerazione rispetto al ciclo singolo tende al rapporto fra i due cicli di clock, e se gli stadi fossero perfettamente bilanciati tenderebbe al numero di stadi. Nell'esempio tende a $800/200 = 4$, non a 5, perché gli stadi del register file usano solo 100 dei 200 ps.

L'ISA RISC-V è pensata per la pipeline: le istruzioni hanno tutte 32 bit (il fetch è sempre uguale), i campi dei registri stanno sempre nello stesso posto (si leggono i registri mentre si decodifica), gli operandi in memoria compaiono solo in `lw` e `sw` (l'indirizzo si calcola in EX e la memoria si usa in MEM) e gli accessi sono allineati.

4.4 I conflitti: strutturali e sui dati

Un **conflitto** (*hazard*) è una situazione in cui l'istruzione successiva non può avanzare nel ciclo previsto. Ce ne sono tre tipi.

Conflitti strutturali. Due istruzioni vogliono la stessa risorsa nello stesso ciclo. Con una sola memoria per istruzioni e dati, il fetch di un'istruzione e la `lw` di tre istruzioni prima si scontrerebbero: per questo la pipeline ha due memorie (in pratica due cache, una per le istruzioni e una per i dati). Il register file è usato in ID (lettura) e in WB (scrittura) da due istruzioni diverse nello stesso ciclo: si risolve scrivendo nella prima metà del ciclo e leggendo nella seconda, così un'istruzione in ID legge il valore appena scritto da quella in WB.

Conflitti sui dati. Un'istruzione ha bisogno di un risultato che una precedente non ha ancora scritto (*read after write*). Nel frammento

```
add  x1, x2, x3
sub  x4, x1, x5
and  x6, x1, x7
or   x8, x1, x9
```

`add` scrive `x1` in WB, al ciclo 5, ma `sub` lo legge in ID al ciclo 3 e `and` al ciclo 4: leggerebbero il valore vecchio. `or` lo legge al ciclo 5, nella seconda metà, e lo trova giusto. Senza altri accorgimenti la pipeline deve **fermarsi** (*stall*): l'istruzione dipendente resta in ID finché il dato è scritto, e negli stadi successivi passano delle **bolle**, istruzioni nulle che non fanno niente. Un'istruzione che dipende da quella immediatamente precedente aspetta 2 cicli, una che dipende da quella di due posti prima 1 ciclo, da tre posti prima in poi nessuno.

4.5 Il forwarding e lo stallo load-use

Il risultato di `add` in realtà esiste già alla fine di EX, nel registro EX/MEM: basta portarlo direttamente all'ingresso dell'ALU dell'istruzione successiva, senza aspettare che passi dal register file. È il **forwarding** (o *bypassing*). Due multiplexer davanti agli ingressi dell'ALU scelgono fra il valore letto dal register file (00), quello in EX/MEM (10) e quello in MEM/WB (01), comandati dall'**unità di forwarding** con queste condizioni.

Conflitto in EX (dall'istruzione precedente):

$$\text{se } \mathrm{EX/MEM.RegWrite} \wedge \mathrm{EX/MEM.rd} \neq 0 \wedge \mathrm{EX/MEM.rd} = \mathrm{ID/EX.rs1} \text{ allora } \mathrm{ForwardA} = 10,$$

e lo stesso con `rs2` per ForwardB.

Conflitto in MEM (da due istruzioni prima): se MEM/WB.RegWrite, MEM/WB.rd ≠ 0, MEM/WB.rd = ID/EX.rs1 **e non** c'è già il conflitto in EX sullo stesso registro, allora ForwardA = 01 (e analogamente per `rs2`).

La condizione rd ≠ 0 serve perché `x0` vale sempre 0: un'istruzione che "scrive" `x0` non deve propagare il suo risultato. La clausola "non c'è già il conflitto in EX" serve per sequenze come

```
add  x1, x1, x2
add  x1, x1, x3
add  x1, x1, x4
```

in cui per la terza istruzione sia EX/MEM sia MEM/WB contengono un valore di `x1`: quello giusto è il più recente, in EX/MEM.

Lo stallo load-use. Il forwarding non basta quando il dato viene da una `lw`: è pronto solo alla fine di MEM, mentre l'istruzione successiva lo vorrebbe all'inizio di EX nello stesso ciclo. Serve un ciclo di stallo, e poi il dato arriva con il forwarding da MEM/WB. Lo inserisce l'**unità di rilevamento dei conflitti** (*hazard detection unit*), che lavora in ID:

$$\text{se } \mathrm{ID/EX.MemRead} \wedge (\mathrm{ID/EX.rd} = \mathrm{IF/ID.rs1} \vee \mathrm{ID/EX.rd} = \mathrm{IF/ID.rs2}) \text{ allora stallo.}$$

Per fermare la pipeline si impedisce la scrittura del PC e di IF/ID (le due istruzioni dietro restano dove sono) e si azzerano i segnali di controllo che entrano in ID/EX, così in EX passa una bolla.

Riordinare il codice. Il compilatore può spesso evitare lo stallo load-use mettendo fra la `lw` e l'istruzione che usa il dato un'istruzione indipendente. È un esempio di come l'hardware e il compilatore lavorino insieme sulle prestazioni (esercizio 3).

4.6 I conflitti sul controllo e la previsione dei salti

Il terzo tipo di conflitto riguarda il PC. Dopo una `beq` la pipeline deve leggere l'istruzione successiva, ma non sa ancora quale: nella pipeline di base la decisione è presa in MEM (il confronto in EX, la scrittura del PC dopo), e se il salto è preso le tre istruzioni lette nel frattempo vanno buttate. Le soluzioni sono quattro.

Stallo. Si aspetta la decisione: si perdono sempre dei cicli, anche quando il salto non è preso.

Anticipare la decisione. Si spostano in ID il sommatore dell'indirizzo di destinazione e un comparatore di uguaglianza fra i due registri (più veloce dell'ALU: basta lo XOR bit a bit e una NOR). Il salto si decide in ID e si perde al massimo un ciclo. Il prezzo: in ID servono anche il forwarding e gli stalli per gli operandi del confronto.

Se la `beq` usa il risultato dell'istruzione aritmetica immediatamente precedente serve 1 ciclo di stallo, se usa quello di una `lw` immediatamente precedente ne servono 2.

Previsione statica. Si prevede che il salto non sia preso e si continua a leggere in sequenza; se poi il salto è preso, l'istruzione letta si scarta (*flush*: il registro IF/ID viene azzerato). Si perde un ciclo solo quando la previsione è sbagliata. La previsione "non preso" funziona bene per i salti in testa ai cicli, che escono di rado, male per quelli in fondo ai cicli, che sono quasi sempre presi: un'alternativa è prevedere presi i salti all'indietro e non presi quelli in avanti. Nell'architettura MIPS delle slide esiste anche il **salto ritardato** (*delayed branch*): l'istruzione dopo il salto viene eseguita sempre, e il compilatore ci mette un'istruzione utile e indipendente (*delay slot*). RISC-V non ha delay slot.

Previsione dinamica. Si usa la storia del salto: una **branch history table** (BHT), indirizzata dai bit bassi del PC e letta in IF, contiene per ogni salto l'esito recente. Con **un bit** si prevede l'ultimo esito, e un ciclo sbaglia due volte a ogni esecuzione completa: all'uscita, e alla prima iterazione della volta successiva. Con **due bit** (un contatore saturato con quattro stati: fortemente preso, debolmente preso, debolmente non preso, fortemente non preso) la previsione cambia solo dopo due errori consecutivi, e il ciclo sbaglia solo all'uscita: per un ciclo di 10 iterazioni si passa dall'80% al 90% di previsioni corrette. La BHT dice *se* saltare ma non *dove*: per non aspettare il calcolo dell'indirizzo, un **branch target buffer** (BTB) memorizza le destinazioni dei salti già incontrati.

Con un salto ogni 5-6 istruzioni, e pipeline sempre più lunghe, la previsione è decisiva per le prestazioni: la penalità di un errore cresce con il numero di stadi che precedono la decisione.

4.7 Le eccezioni

Un'**eccezione** è un evento che interrompe il normale flusso del programma; si chiama **interruzione** (*interrupt*) quando viene dall'esterno del processore, come una richiesta di un dispositivo di I/O. Per la pipeline è un conflitto sul controllo: bisogna fermare l'esecuzione, completare le istruzioni precedenti, scartare quella che ha causato l'eccezione e le successive, salvare l'indirizzo dell'istruzione (in RISC-V nel registro **SEPC**) e la causa (nel registro **SCAUSE**), e saltare a un indirizzo fisso dove si trova il gestore del sistema operativo.

Evento	Stadio	Sincrono?
Istruzione non definita	ID	sì
Page fault, errore di accesso alla memoria	IF, MEM	sì
Overflow (nelle ISA che lo segnalano, come MIPS)	EX	sì
Richiesta di un dispositivo di I/O	qualsiasi	no
Malfunzionamento dell'hardware	qualsiasi	no

Nello stesso ciclo ci possono essere più eccezioni, in stadi diversi: l'hardware dà la precedenza a quella dell'istruzione più vecchia, cioè più avanti nella pipeline. Le eccezioni gestite in modo che tutte le istruzioni precedenti siano completate e nessuna successiva abbia modificato lo stato si dicono **precise**.

> **Lo schema dell'appunto a mano**
>
> L'appunto a mano disegna il datapath completo della fine del capitolo: i registri IF/ID, ID/EX, EX/MEM, MEM/WB; il comparatore di uguaglianza in ID per decidere i salti, con una sua unità di forwarding; l'unità di forwarding in EX che comanda i multiplexer davanti all'ALU; l'unità di rilevamento dei conflitti in ID; e, in ingresso al multiplexer del PC, l'indirizzo fisso del gestore delle eccezioni (nello schema quello MIPS delle slide, `8000 0180` esadecimale).

4.8 Più istruzioni per ciclo

La pipeline avvicina il CPI a 1. Per scendere sotto 1 bisogna avviare più istruzioni per ciclo (*multiple issue*); in questo caso si usa spesso il suo inverso, l'**IPC** (istruzioni per ciclo). Le strade sono due.

Avvio multiplo statico (VLIW). È il compilatore a raggruppare le istruzioni in **pacchetti** che partono insieme. Nell'esempio del corso il pacchetto ha due posti: uno per un'istruzione aritmetico-logica o un salto, uno per una `lw` o una `sw`. Servono più porte di lettura e scrittura nel register file e un sommatore in più per gli indirizzi. Il

compilatore deve rispettare le dipendenze: in un pacchetto non ci possono essere due istruzioni dipendenti, il risultato di un'istruzione aritmetica si può usare nel pacchetto successivo, quello di una `lw` solo due pacchetti dopo; se non trova un'istruzione per un posto, ci mette una `nop`.

Per riempire i pacchetti si usa lo **srotolamento dei cicli** (*loop unrolling*): si copia più volte il corpo del ciclo, si aggiorna il puntatore una volta sola e si usano registri diversi per le varie copie. Così diminuiscono le istruzioni di controllo del ciclo e ci sono più istruzioni indipendenti da accoppiare (esercizio 7). L'uso di registri diversi elimina le **antidipendenze** fra le copie: è una ridenominazione dei registri fatta dal compilatore.

Avvio multiplo dinamico (superscalare). È l'hardware a decidere, durante l'esecuzione, quante e quali istruzioni avviare. Le istruzioni vengono lette e decodificate in ordine e messe in una coda; vengono **eseguite fuori ordine** (*out of order*), appena gli operandi sono disponibili, in unità funzionali con le loro stazioni di prenotazione; i risultati vengono scritti nei registri e in memoria di nuovo **in ordine** (*commit*), attraverso un buffer di riordino, così le eccezioni restano precise e le istruzioni eseguite per una previsione sbagliata (esecuzione **speculativa**) si possono annullare.

Con l'esecuzione fuori ordine non contano più solo le dipendenze vere. Fra due istruzioni i e j, con i prima di j:

- **dipendenza vera** (*read after write*, RAW): j legge un registro scritto da i;
- **antidipendenza** (*write after read*, WAR): j scrive un registro letto da i;
- **dipendenza di uscita** (*write after write*, WAW): j e i scrivono lo stesso registro.

Le dipendenze vere vengono dal flusso dei dati dell'algoritmo e non si possono togliere. Le altre due nascono solo perché i registri dell'ISA sono pochi e vengono riusati: sono in realtà conflitti strutturali, e l'hardware li elimina con la **ridenominazione dei registri** (*register renaming*), usando un insieme più grande di registri fisici.

I limiti: la tabella dei processori Intel delle slide mostra che dal Pentium 4 (fino a 31 stadi e 3,6 GHz, oltre 100 W) si è tornati a pipeline più corte e a più core per chip. Le pipeline lunghe e l'avvio multiplo sono frenati dal consumo e dal calore (*power wall*), dal parallelismo limitato presente nei programmi e dalla latenza della memoria.

Esempi ed esercizi svolti

> **Aggiunta in revisione · fonte: esercizi nuovi sul modello delle slide (Pipelined Architecture, Control Hazards, Multiple Issue); conteggi controllati con `strumenti/pipe.py` e `strumenti/verifica_ampliamento.py` (ADC-04, ADC-06, ADC-07)**

Esercizio 1. Ciclo singolo o pipeline? Memorie 250 ps, ALU 150 ps, register file 100 ps. Si confrontino il ciclo singolo e la pipeline a 5 stadi su 1000 istruzioni senza conflitti.

Ciclo singolo: l'istruzione più lunga è `lw`, $250 + 100 + 150 + 250 + 100 = 850$ ps; il programma dura $1000 \times 850 = 850$ ns. Pipeline: il ciclo è quello dello stadio più lento, 250 ps; il programma dura $(5 + 999) \times 250 = 251$ ns. L'accelerazione è $850/251 \approx 3{,}39$. Il limite per $n \to \infty$ è $850/250 = 3{,}4$, lontano da 5 perché gli stadi sono sbilanciati: ID, EX e WB lavorano 100 o 150 ps su 250.

Esercizio 2. I segnali di controllo di `lw` e `beq`. Perché ALUSrc vale 1 in `lw` e 0 in `beq`, e perché ALUOp è diverso?

`lw x5, 8(x10)` somma il registro `x10` e l'immediato 8: il secondo operando dell'ALU è l'immediato, ALUSrc = 1, e l'operazione è una somma, ALUOp = 00. `beq x5, x6, L` confronta due registri: il secondo operando è un registro, ALUSrc = 0, e il confronto si fa con una sottrazione che accende Zero se il risultato è nullo, ALUOp = 01. L'immediato di `beq` serve, ma al sommatore della destinazione, non all'ALU.

Esercizio 3. Stalli e riordino. Si contino i cicli del frammento, con e senza forwarding, e lo si riordini per togliere gli stalli.

```
lw   t0, 0(a0)
lw   t1, 4(a0)
add  t2, t0, t1
```

```
sw   t2, 8(a0)
addi a0, a0, 12
```

Senza forwarding. `add` legge `t1` in ID e deve aspettare che la seconda `lw` lo scriva in WB: 2 stalli. `sw` legge `t2`, prodotto da `add` subito prima: altri 2 stalli. In tutto $5 + 4 + 4 = 13$ cicli.

Con forwarding. Resta solo lo stallo load-use fra la seconda `lw` e `add`: 10 cicli. Il diagramma:

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
`lw t0`	IF	ID	EX	MEM	WB					
`lw t1`		IF	ID	EX	MEM	WB				
`add`			IF	ID	ID	EX	MEM	WB		
`sw`				IF	IF	ID	EX	MEM	WB	
`addi`						IF	ID	EX	MEM	WB

Al ciclo 5 in EX passa una bolla; `t1` arriva all'ALU di `add` al ciclo 6 dal registro MEM/WB.

Riordino. `addi` non dipende da nessuna delle istruzioni precedenti tranne che per `a0`, che le due `lw` leggono prima. La si sposta fra la seconda `lw` e `add`, correggendo lo spiazzamento della `sw`, che ora vede `a0` già aumentato di 12:

```
lw   t0, 0(a0)
lw   t1, 4(a0)
addi a0, a0, 12
add  t2, t0, t1
sw   t2, -4(a0)
```

Con il forwarding non ci sono più stalli: 9 cicli, il minimo per 5 istruzioni. Senza forwarding servono 12 cicli.

Esercizio 4. Una catena di dipendenze.

```
add  s1, s2, s3
sub  s4, s1, s5
and  s6, s1, s4
or   s7, s6, s2
sw   s7, 0(a0)
```

Ogni istruzione dipende da quella immediatamente precedente. Senza forwarding: 2 stalli per ciascuna delle ultime quattro, $9 + 8 = 17$ cicli. Con il forwarding nessuno stallo, perché nessun dato viene da una `lw`: 9 cicli. Il guidato 1 mostra da dove arriva ogni operando.

Esercizio 5. Salti decisi in ID. Quanti stalli in queste due coppie, con forwarding e salti decisi in ID?

```
lw   t0, 0(a0)          addi t1, t1, 1
beq  t0, zero, fine     bne  t1, t2, ciclo
```

La `beq` confronta in ID, un ciclo prima di quando un'istruzione aritmetica userebbe l'operando in EX. Il valore di `lw` è pronto alla fine di MEM: se `lw` è in MEM al ciclo c, la `beq` può essere in ID solo al ciclo $c + 1$, cioè 2 stalli. Il valore di `addi` è pronto alla fine di EX: 1 stallo. In un ciclo conviene quindi mettere fra l'aggiornamento del contatore e il salto un'istruzione indipendente.

Esercizio 6. Il CPI con stalli e salti. In un programma il 25% delle istruzioni sono `lw`, e il 40% di queste è seguito da un'istruzione che usa il dato; il 15% sono salti, presi nel 60% dei casi. La pipeline ha il forwarding e prevede "non preso". Calcolare il CPI con salti decisi in ID e in MEM, e con un predittore dinamico corretto nel 90% dei casi (e un BTB) con decisione in ID.

Ogni stallo load-use costa 1 ciclo: $0{,}25 \times 0{,}4 = 0{,}1$ cicli per istruzione. Decisione in ID: un salto preso costa 1 ciclo, $0{,}15 \times 0{,}6 \times 1 = 0{,}09$; $\mathrm{CPI} = 1 + 0{,}1 + 0{,}09 = 1{,}19$. Decisione in MEM: un salto preso costa 3 cicli, $0{,}15 \times 0{,}6 \times 3 = 0{,}27$; $\mathrm{CPI} = 1{,}37$. Predittore dinamico: si paga 1 ciclo solo sugli errori, $0{,}15 \times 0{,}1 = 0{,}015$; $\mathrm{CPI} = 1{,}115$.

Esercizio 7. VLIW e srotolamento. Il ciclo `b[i] = 2 * a[i]` su n elementi (n pari), con `a0` = indirizzo di `a`, `a1` = indirizzo di `b`, `a2` = indirizzo di fine di `a`, va schedulato su un processore a due vie (aritmetica o salto | `lw` o `sw`).

```
lp:  lw   t0, 0(a0)
     slli t0, t0, 1
     sw   t0, 0(a1)
     addi a0, a0, 4
     addi a1, a1, 4
     bne  a0, a2, lp
```

Senza srotolamento. `slli` usa il dato della `lw`: almeno due pacchetti dopo. `sw` usa `slli`: un pacchetto dopo. Le due `addi` si possono anticipare, correggendo lo spiazzamento della `sw`:

```
lp:  addi a0, a0, 4     |  lw   t0, 0(a0)
     addi a1, a1, 4     |  nop
     slli t0, t0, 1     |  nop
     bne  a0, a2, lp    |  sw   t0, -4(a1)
```

Nel primo pacchetto `lw` legge `a0` insieme ad `addi`, quindi il valore vecchio. Sono 6 istruzioni in 4 cicli: IPC 1,5.

Srotolando due volte (copie con `t0` e `t1`, puntatori aggiornati di 8):

```
lp:  addi a0, a0, 8     |  lw   t0, 0(a0)
     addi a1, a1, 8     |  lw   t1, -4(a0)
     slli t0, t0, 1     |  nop
     slli t1, t1, 1     |  sw   t0, -8(a1)
     bne  a0, a2, lp    |  sw   t1, -4(a1)
```

Sono 9 istruzioni in 5 cicli: IPC 1,8, cioè 2,5 cicli per elemento invece di 4. Il secondo `lw` vede `a0` già aumentato di 8, per questo il suo spiazzamento è -4 (cioè $+4$ rispetto al valore vecchio). Senza il secondo registro `t1` le due copie si pesterebbero i piedi: è la ridenominazione fatta dal compilatore.

Esercizio 8. Dipendenze e ridenominazione.

```
mul  t0, t1, t2      # 1
add  t3, t0, t4      # 2
sub  t0, t5, t6      # 3
and  t4, t0, t3      # 4
```

Dipendenze vere: $1 \to 2$ su `t0`, $3 \to 4$ su `t0`, $2 \to 4$ su `t3`. Antidipendenze: 2 legge `t0` che 3 scrive ($2 \to 3$); 2 legge `t4` che 4 scrive ($2 \to 4$). Dipendenza di uscita: 1 e 3 scrivono `t0`. Ridenominando il `t0` scritto da 3 in un registro fisico nuovo `p1` (e il `t4` scritto da 4 in `p2`), la 3 diventa `sub p1, t5, t6` e la 4 `and p2, p1, t3`: la 3 può partire subito, in parallelo alla `mul`, che è lenta.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; soluzioni controllate con `strumenti/pipe.py` e `strumenti/verifica_ampliamento.py` (ADC-04, ADC-06)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. I segnali di forwarding. Per il frammento dell'esercizio 4, dire per ogni istruzione che valori hanno ForwardA e ForwardB quando è in EX.

Passo 1: che cosa c'è negli stadi. Quando l'istruzione i è in EX, la $i-1$ è in MEM (il suo risultato è in EX/MEM) e la $i-2$ in WB (il risultato è in MEM/WB).

Passo 2: `sub s4, s1, s5`. In EX/MEM c'è `add`, con rd = `s1` = rs1: ForwardA = 10. `s5` viene dal register file: ForwardB = 00.

Passo 3: `and s6, s1, s4`. In EX/MEM c'è `sub` (rd = `s4` = rs2): ForwardB = 10. In MEM/WB c'è `add` (rd = `s1` = rs1), e EX/MEM non scrive `s1`: ForwardA = 01.

Passo 4: `or s7, s6, s2`. In EX/MEM c'è `and` (rd = `s6` = rs1): ForwardA = 10. `s2` non è scritto da nessuna delle due: ForwardB = 00.

Passo 5: `sw s7, 0(a0)`. Il dato da scrivere `s7` (rs2) viene da `or` in EX/MEM: ForwardB = 10; `a0` (rs1) dal register file: ForwardA = 00.

Guidato 2. Predittore a 1 e a 2 bit su un ciclo annidato. Il ciclo interno ha 4 iterazioni e viene eseguito molte volte: il suo salto in fondo dà la sequenza preso, preso, preso, non preso, ripetuta. Che percentuale di previsioni corrette danno i due predittori a regime?

Passo 1: un bit. All'inizio di un'esecuzione il bit vale "non preso" (l'ultimo esito). Prima iterazione: previsto non preso, è preso, errore; il bit diventa "preso". Seconda e terza: giuste. Quarta: previsto preso, non preso, errore. Due errori su quattro: 50%.

Passo 2: due bit. A regime il contatore è a "fortemente preso" durante il ciclo; all'uscita sbaglia e scende a "debolmente preso". Alla prima iterazione successiva prevede ancora preso e indovina, tornando a "fortemente preso". Un errore su quattro: 75%.

Passo 3: il confronto. Più il ciclo è corto, più conta il secondo bit: per 10 iterazioni si passa da 80% a 90%, per 4 da 50% a 75%.

Guidato 3. Diagramma con un salto preso. Con forwarding, salti decisi in ID e previsione "non preso", quanti cicli impiega questo frammento se il salto è preso e la destinazione è `fine: sub t3, t0, t1`?

```
     addi t1, t1, 1
     beq  t1, t2, fine
     add  t4, t4, t1
fine: sub  t3, t0, t1
```

Passo 1: lo stallo. `beq` usa `t1` da `addi` immediatamente prima, e la decide in ID: 1 stallo.

Passo 2: il flush. Mentre `beq` è in ID (al secondo tentativo, ciclo 4), in IF c'è `add`. Il salto è preso: `add` si scarta e al ciclo 5 si legge `sub`.

Passo 3: il conto. `sub` è in IF al ciclo 5 e in WB al ciclo 9. In tutto 9 cicli per 3 istruzioni utili: i cicli persi sono uno per lo stallo e uno per il flush.

Riepilogo

Il datapath a ciclo singolo esegue ogni istruzione in un ciclo lungo quanto quella più lenta; l'unità di controllo genera dai campi dell'istruzione i segnali dei multiplexer, delle scritture e dell'ALU, con il controllo dell'ALU a due livelli. La pipeline a cinque stadi IF, ID, EX, MEM, WB, con i registri di pipeline, completa a regime un'istruzione per ciclo: aumenta il throughput, non riduce la latenza, e il ciclo è fissato dallo stadio più lento. I conflitti strutturali si evitano con memorie separate e con il register file che scrive e legge nelle due metà del ciclo. I conflitti sui dati si risolvono con il forwarding da EX/MEM e da MEM/WB; resta lo stallo load-use, inserito dall'unità di rilevamento dei conflitti e spesso evitabile riordinando il codice. I conflitti sul controllo si riducono decidendo i salti in ID e con la previsione statica o dinamica (BHT a uno o due bit, BTB). Le eccezioni sono conflitti sul controllo gestiti salvando l'indirizzo e la causa. Per avviare più istruzioni per ciclo ci sono il VLIW, in cui il compilatore forma i pacchetti aiutandosi con lo srotolamento dei cicli, e i processori superscalari, che eseguono fuori ordine, eliminano antidipendenze e dipendenze di uscita con la ridenominazione e scrivono i risultati in ordine.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (ADC-04, ADC-06, ADC-07)**

Tempi

$$T_{\text{ciclo singolo}} = n \cdot T_{\text{istr. più lenta}}, \qquad T_{pipeline} = (k + n - 1)T_c, \qquad T_c = \max(\text{stadi}), \qquad \text{accelerazione} \to \frac{T_{\text{ciclo singolo}}}{T_c}.$$

$$\text{CPI} = 1 + \sum(\text{frequenza} \times \text{cicli di stallo}), \qquad \text{IPC} = 1/\text{CPI}.$$

Stalli nella pipeline a 5 stadi

Situazione	Senza forwarding	Con forwarding
uso subito dopo un’istruzione aritmetica	2	0
uso due istruzioni dopo	1	0
uso subito dopo una `lw`	2	1
salto in ID subito dopo un’aritmetica	2	1
salto in ID subito dopo una `lw`	2	2
salto preso con previsione “non preso”	1 (decisione in ID), 3 (in MEM)	uguale

Condizioni

Forwarding da EX/MEM: RegWrite ∧ rd ≠ 0 ∧ rd = rs1 (rs2) → 10. Da MEM/WB: RegWrite ∧ rd ≠ 0 ∧ rd = rs1 (rs2) ∧ non dal precedente → 01. Load-use: ID/EX.MemRead ∧ ID/EX.rd ∈ {IF/ID.rs1, IF/ID.rs2} → stallo (PC e IF/ID fermi, controlli azzerati).

Segnali di controllo

	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R	0	0	1	0	0	0	10
`lw`	1	1	1	1	0	0	00
`sw`	1	X	0	0	1	0	00
`beq`	0	X	0	0	0	1	01

Termini per l’orale

Termine	In una riga
Throughput, latenza	istruzioni completate per unità di tempo; durata di una istruzione
Registro di pipeline	conserva dati e controlli fra due stadi
Hazard strutturale, sui dati, sul controllo	risorsa contesa; dato non pronto; PC non noto
Stallo, bolla	fermare le istruzioni dietro; istruzione nulla che avanza
Forwarding	portare un risultato all’ALU prima della scrittura nei registri
Load-use	uso del dato di una `lw` subito dopo: 1 stallo
Flush	scartare le istruzioni lette per errore
BHT, BTB	tabella degli esiti dei salti; tabella delle destinazioni
Eccezione precisa	prima completate, dopo nessun effetto
VLIW, superscalare	avvio multiplo deciso dal compilatore; dall’hardware
RAW, WAR, WAW	dipendenza vera, antidipendenza, di uscita
Register renaming	registri fisici in più per eliminare WAR e WAW

Domande che tornano spesso

- Disegnare il datapath a ciclo singolo e spiegare i segnali di controllo per `lw`, `sw`, `beq` e tipo R.

- Perché la pipeline aumenta il throughput ma non riduce la latenza? Formula dell’accelerazione.
- Tipi di conflitti e soluzioni; condizioni del forwarding e dello stallo load-use.
- Come si riducono i conflitti sul controllo? Predittori a uno e due bit.
- Come gestisce le eccezioni la pipeline?
- VLIW e superscalare: differenze; srotolamento dei cicli; dipendenze e ridenominazione.

5 · La gerarchia di memoria, la cache e l'input/output

In questo modulo. Si usano le prestazioni e il CPI del modulo 1 e la pipeline del modulo 4. Il processore è molto più veloce della memoria principale, e la distanza cresce da decenni. Si vede come la si colma con una gerarchia di memorie sempre più grandi e lente, che funziona grazie al principio di località. Il cuore del modulo è la cache: come si trova un dato, le funzioni di corrispondenza (diretta, associativa a insiemi, completamente associativa), la dimensione, la sostituzione, le politiche di scrittura, le cause dei miss, l'effetto sulle prestazioni, le cache a più livelli e il collegamento con la DRAM. Dopo un cenno alla memoria virtuale, l'ultima parte tratta l'input/output: bus, polling, interruzioni e DMA. Alla fine del modulo si sa scomporre un indirizzo in tag, indice e offset, simulare a mano una sequenza di accessi, dimensionare una cache e calcolare il tempo medio di accesso e il CPI con gli stalli di memoria.

> **Aggiunta in revisione · fonte: slide del corso (Gerarchia di memoria, Input-Output); Patterson e Hennessy, *Struttura e progetto dei calcolatori*, edizione RISC-V, cap. 5 e 6 (ADC-05, ADC-01)**
>
> Il modulo è scritto in revisione dal materiale del corso. Le sequenze di accessi sono simulate con `strumenti/cache.py`, i conti controllati con `strumenti/verifica_ampliamento.py`.

5.1 Perché una gerarchia di memoria

Le prestazioni dei processori sono cresciute per molti anni di circa il 55% all'anno, quelle della DRAM di circa il 7%: un accesso alla memoria principale costa oggi centinaia di cicli di clock. Una memoria insieme grande, veloce ed economica non esiste. Le tecnologie disponibili sono:

- **SRAM**: 6 transistor per bit, accesso intorno al nanosecondo, costosa e poco densa; è *statica*, conserva il dato finché è alimentata. Si usa per le cache.
- **DRAM**: 1 transistor e un condensatore per bit, accesso di qualche decina di nanosecondi, densa ed economica; è *dinamica*, il condensatore si scarica e il contenuto va rinfrescato periodicamente (ogni pochi millisecondi). Si usa per la memoria principale.
- **Memoria secondaria** (dischi magnetici e flash): enorme e permanente, ma con tempi da microsecondi a millisecondi.

La soluzione è una **gerarchia**: registri, cache di primo livello (L1, separate per istruzioni e dati), cache di secondo e terzo livello, memoria principale, memoria secondaria. Ogni livello è più grande, più lento e meno caro per bit del precedente, e contiene un sottoinsieme dei dati del livello sotto. Se quasi tutti gli accessi trovano il dato nei livelli alti, il sistema appare veloce quasi come il primo livello e grande come l'ultimo.

Questo funziona per il **principio di località**. **Località temporale**: un dato appena usato verrà probabilmente usato di nuovo presto (le variabili di un ciclo, le istruzioni del ciclo stesso); conviene tenere vicino al processore i dati usati di recente. **Località spaziale**: dopo un indirizzo verranno probabilmente usati quelli vicini (le istruzioni in sequenza, gli elementi di un array); conviene spostare verso l'alto interi blocchi di dati contigui.

Chi gestisce i passaggi fra i livelli: fra registri e memoria il compilatore; fra cache e memoria principale l'hardware (il controllore della cache); fra memoria principale e disco il sistema operativo, con la memoria virtuale (5.8).

5.2 Terminologia

Il **blocco** (o **linea**) è l'unità di informazione che si trasferisce fra due livelli. Se il dato cercato è nel livello superiore si ha un **hit**, altrimenti un **miss**, e il blocco va preso dal livello inferiore.

- **Hit rate**: frazione degli accessi che sono hit; **miss rate** $= 1 -$ hit rate.
- **Hit time**: tempo per accedere al livello superiore, compreso il tempo per capire se è un hit.
- **Miss penalty**: tempo per portare il blocco dal livello inferiore, sistemarlo e consegnare il dato al processore.

Di solito l'hit time è molto minore della miss penalty, e il **tempo medio di accesso** è

$$\text{AMAT} = \text{hit time} + \text{miss rate} \times \text{miss penalty}.$$

5.3 La cache a corrispondenza diretta

La cache deve rispondere a due domande: il dato è presente? Se sì, dove? Nella cache **a corrispondenza diretta** (*direct mapped*) ogni blocco della memoria può stare in un solo posto:

$$\text{posizione} = (\text{indirizzo del blocco}) \bmod (\text{numero di blocchi della cache}).$$

Se il numero di blocchi è una potenza di 2, il modulo sono semplicemente i bit bassi dell'indirizzo del blocco. Molti blocchi di memoria condividono la stessa posizione: per sapere quale c'è davvero, ogni posizione conserva anche i bit alti dell'indirizzo, il **tag**, e un **bit di validità** che dice se la posizione contiene qualcosa (all'accensione sono tutte vuote).

Un indirizzo di memoria si divide così in tre campi:

tag	indice	offset nel blocco
bit alti, confrontati con quelli memorizzati	sceglie la posizione	sceglie la parola e il byte

Con blocchi di 2^m parole da 4 byte l'offset ha $m + 2$ bit; con 2^n blocchi l'indice ha n bit; con indirizzi di 32 bit il tag ha $32 - n - m - 2$ bit. Una lettura usa l'indice per scegliere la posizione, confronta il tag e controlla il bit di validità: se coincidono è un hit e l'offset sceglie la parola.

La dimensione effettiva della cache comprende anche tag e bit di validità:

$$\text{bit totali} = 2^n \times (\text{bit del blocco} + \text{bit del tag} + 1).$$

Con 16 KB di dati in blocchi di 4 parole (esempio delle slide): 2^{10} blocchi, tag di 18 bit, $2^{10} \times (128 + 18 + 1) = 147$ Kbit, cioè circa 18,4 KB per 16 KB di dati.

La dimensione del blocco. Blocchi più grandi sfruttano meglio la località spaziale e riducono il miss rate, ma a parità di capacità diminuisce il numero di blocchi: oltre un certo punto i blocchi si contendono le posizioni e il miss rate risale. In più cresce la miss penalty, perché c'è più da trasferire. Per limitarla si può far ripartire il processore appena arriva la parola richiesta (*early restart*) o farla arrivare per prima (*requested word first*).

5.4 Le scritture

In una lettura il controllo del tag e la lettura del dato si possono fare insieme. Con le scritture bisogna decidere come tenere coerenti cache e memoria.

Write-through. Ogni scrittura aggiorna sia la cache sia la memoria. È semplice e la memoria è sempre aggiornata, ma ogni scrittura costerebbe un accesso alla memoria: si aggiunge un **write buffer**, una coda in cui il processore lascia le scritture e prosegue, mentre la memoria le smaltisce. Il processore si ferma solo se il buffer è pieno.

Write-back. Si scrive solo nella cache, e il blocco viene ricopiato in memoria quando viene sostituito. Serve un bit in più per blocco, il **dirty bit**, che dice se il blocco è stato modificato: i blocchi non modificati si possono sostituire senza riscriverli. Riduce molto il traffico verso la memoria quando un blocco viene scritto più volte, ma per un certo tempo la memoria contiene dati vecchi.

Scrittura con miss. Con **write allocate** si porta il blocco in cache e poi lo si scrive (di solito insieme al write-back); con **no-write allocate** si scrive direttamente nel livello inferiore senza caricare il blocco (di solito con il write-through).

Miss in lettura. Si ferma la pipeline, si legge il blocco dal livello inferiore (riscrivendo prima, con il write-back, il blocco sostituito se è sporco), lo si scrive in cache e si ripete l'accesso. Le cache più evolute sono **non bloccanti**: continuano a servire gli hit mentre un miss è in corso (*hit under miss*).

5.5 Le cause dei miss e le cache associative

I miss hanno tre cause (le “tre C”):

- **obbligatori** (*compulsory*): il primo accesso a un blocco; si riducono con blocchi più grandi;
- **di capacità**: la cache non contiene tutti i blocchi usati dal programma; si riducono con una cache più grande;
- **di conflitto**: più blocchi usati insieme vanno nella stessa posizione, anche se altrove c’è posto; si riducono aumentando l’associatività.

Per ridurre i conflitti si rende più flessibile il posizionamento dei blocchi. All’estremo opposto della corrispondenza diretta c’è la cache **completamente associativa**: un blocco può stare in qualsiasi posizione, ma per trovarlo bisogna confrontare il tag con tutti i tag della cache, con un comparatore per ogni blocco. Il compromesso è la cache **associativa a n vie** (*set associative*): la cache è divisa in **insiemi** di n blocchi; un blocco va in un insieme preciso,

$$\text{insieme} = (\text{indirizzo del blocco}) \bmod (\text{numero di insiemi}),$$

ma al suo interno in una qualsiasi delle n vie. L’indice sceglie l’insieme, e si confrontano in parallelo gli n tag. A parità di capacità, raddoppiando le vie si dimezza il numero di insiemi: l’indice perde un bit e il tag ne guadagna uno. La corrispondenza diretta è il caso a una via, la completamente associativa quello con un solo insieme.

Il prezzo dell’associatività: n comparatori, un multiplexer per scegliere la via (più ritardo sull’hit time), e una politica di **sostituzione** quando l’insieme è pieno. La più usata è **LRU** (*least recently used*): si sostituisce il blocco non usato da più tempo. Per due vie basta un bit per insieme; con più vie l’LRU esatto è costoso e si usano approssimazioni (come NRU, *not recently used*) o una scelta casuale.

5.6 Le prestazioni della cache

Gli stalli dovuti alla memoria si aggiungono al CPI della pipeline:

$$\text{CPI} = \text{CPI}_{base} + \frac{\text{accessi}}{\text{istruzione}} \times \text{miss rate} \times \text{miss penalty},$$

separando di solito le istruzioni (un accesso per istruzione) dai dati (un accesso per ogni `lw` o `sw`). Due osservazioni importanti. Se il processore diventa più veloce, la miss penalty *in cicli* cresce, perché la memoria non accelera: il peso degli stalli aumenta (guidato 3). E ridurre il CPI di base serve sempre meno, perché la parte dovuta ai miss resta.

Cache a più livelli. Una cache di secondo livello (L2), più grande e più lenta della L1 ma molto più veloce della memoria, riduce la miss penalty della L1: un miss in L1 che trova il dato in L2 costa poche decine di cicli invece di centinaia. I due livelli hanno obiettivi diversi: la L1 deve avere un hit time minimo (è piccola, spesso a poche vie), la L2 un miss rate basso (è grande, più associativa, con blocchi più grandi). Si distingue il miss rate **locale** della L2 (miss della L2 diviso accessi alla L2, che sono solo i miss della L1) da quello **globale** (miss della L2 diviso tutti gli accessi): il locale è molto più alto del globale, perché alla L2 arrivano solo gli accessi “difficili”. Con due livelli

$$\text{AMAT} = \text{hit time}_{L1} + \text{miss rate}_{L1} \times (\text{hit time}_{L2} + \text{miss rate locale}_{L2} \times \text{miss penalty}_{L2}).$$

Il software. Le prestazioni della cache dipendono anche da come il programma accede ai dati: scorrere una matrice per righe (come è memorizzata in C) sfrutta la località spaziale, scorrerla per colonne no. Gli algoritmi possono essere riorganizzati a blocchi (*blocking*) per riusare i dati finché sono in cache.

5.7 Cache e DRAM

La miss penalty dipende da come la cache è collegata alla DRAM. In una DRAM l’indirizzo si dà in due parti: prima la riga (segnale RAS), che viene copiata in un buffer, poi la colonna (CAS); parole successive della stessa riga si leggono molto più in fretta della prima. Le SDRAM sono sincrone con il bus; le DDR trasferiscono un dato su entrambi i fronti del clock, e le generazioni successive (DDR2, DDR3, DDR4) aumentano la velocità di trasferimento e il prefetch interno, abbassando la tensione.

Il modello delle slide per un blocco di più parole su un bus largo una parola: 1 ciclo di bus per mandare l’indirizzo, un certo numero di cicli per ogni accesso alla DRAM, 1 ciclo per trasferire una parola (il

trasferimento di una parola si sovrappone alla lettura della successiva, tranne l'ultimo). Si confrontano tre organizzazioni:

- ogni parola richiede un accesso completo alla DRAM;
- le parole del blocco stanno nella stessa riga, e dopo la prima basta l'accesso di colonna, più breve;
- la memoria è **interallacciata** (*interleaved*) in banchi: le parole del blocco stanno in banchi diversi, letti in parallelo, e poi trasferite una per ciclo.

La **larghezza di banda** è il numero di byte trasferiti per ciclo di bus durante un miss. L'esercizio 6 fa i conti: lo scopo è scegliere DRAM e bus adatti alla dimensione dei blocchi della cache.

5.8 Un cenno alla memoria virtuale

Fra memoria principale e disco la gerarchia è gestita dal sistema operativo con la **memoria virtuale**. Ogni programma usa indirizzi **virtuali**, divisi in pagine (tipicamente di 4 KB); il sistema operativo decide in quale pagina **fisica** della memoria principale sta ogni pagina virtuale, oppure se è sul disco. L'indirizzo virtuale si divide in numero di pagina virtuale e offset nella pagina; la traduzione sostituisce il numero di pagina virtuale con quello fisico, usando la **tabella delle pagine** del processo. Se la pagina non è in memoria si ha un **page fault**, un'eccezione gestita dal sistema operativo, che la carica dal disco: costa milioni di cicli, per questo la corrispondenza è completamente associativa, la sostituzione approssima LRU e la scrittura è sempre write-back. La memoria virtuale serve anche a proteggere i processi uno dall'altro, perché ciascuno vede solo le sue pagine: è la virtualizzazione del modulo 1 applicata alla memoria.

Per non leggere la tabella delle pagine in memoria a ogni accesso, il processore tiene le traduzioni recenti in una piccola cache, il **TLB** (*translation lookaside buffer*), che nella figura della gerarchia delle slide compare accanto alle cache L1 (ITLB e DTLB).

5.9 L'input/output

I dispositivi di I/O sono molto diversi per comportamento (ingresso, uscita, memorizzazione), per interlocutore (una persona o una macchina) e per velocità: fra una tastiera e una scheda di rete ci sono circa otto ordini di grandezza. Le misure di prestazione sono il **throughput** (quanti dati o quante operazioni per unità di tempo) e il **tempo di risposta** (la latenza di una singola operazione, decisiva nei sistemi in tempo reale).

I bus. Un bus è un collegamento condiviso fra più sottosistemi. È versatile ed economico, ma è un collo di bottiglia, e la sua velocità è limitata dalla lunghezza e dal numero di dispositivi. Si distinguono il bus processore-memoria (corto, veloce, proprietario), i bus di I/O (lunghi, più lenti, standard, come USB e SATA) e i bus di collegamento (*backplane*, come PCI Express). Un bus **sincrono** ha un clock fra le linee di controllo e un protocollo fisso rispetto al clock: poca logica e alta velocità, ma tutti i dispositivi devono seguire lo stesso clock e il bus deve essere corto per evitare lo sfasamento del clock (*clock skew*). Un bus **asincrono** non ha clock e usa un protocollo di **handshaking** con linee di controllo apposite (richiesta, conferma, dato pronto): si adatta a dispositivi di velocità diverse ed è più lento. La tendenza è passare da bus paralleli e larghi a collegamenti seriali punto a punto molto veloci (SATA al posto di ATA parallelo, PCIe al posto di PCI).

Comandare i dispositivi. Il processore comunica con un dispositivo attraverso i suoi registri (di stato, di controllo, dei dati), con istruzioni speciali di I/O oppure, come in RISC-V, con l'**I/O mappato in memoria**: a ogni dispositivo è assegnata una zona dello spazio degli indirizzi, e `lw` e `sw` su quegli indirizzi diventano letture e comandi per il dispositivo. Il sistema operativo fa da intermediario, perché i dispositivi sono condivisi fra i programmi e vanno protetti.

Sapere quando il dispositivo è pronto. Ci sono tre modi.

- **Polling**: il processore legge periodicamente il registro di stato del dispositivo. È semplice e il processore ha tutto sotto controllo, ma spreca cicli quando il dispositivo è lento o inattivo.
- **Interruzioni**: è il dispositivo a segnalare con un'interruzione che ha bisogno di attenzione. L'interruzione è asincrona rispetto alle istruzioni; il processore la serve fra un'istruzione e l'altra, come un'eccezione (4.7). Servono un modo per identificare il dispositivo e dei livelli di priorità: il gestore maschera le interruzioni di priorità uguale o minore, salva lo stato, riabilita quelle più urgenti e alla fine ripristina tutto.
- **DMA** (*direct memory access*): per i dispositivi veloci, come i dischi, anche un'interruzione per ogni parola costerebbe troppo. Il processore programma il controllore DMA (indirizzo del dispositivo, operazione,

indirizzo in memoria, numero di byte), il controllore trasferisce da solo l'intero blocco contendendosi il bus con il processore, e alla fine manda una sola interruzione.

Con la DMA nascono due problemi. **Dati vecchi**: se un blocco di memoria è anche in cache, un ingresso DMA aggiorna la memoria ma non la cache, e con una cache write-back un'uscita DMA può leggere dalla memoria dati non ancora riscritti. Si risolve facendo passare l'I/O dalla cache (costoso), facendo invalidare o riscrivere dal sistema operativo i blocchi interessati (*cache flush*), o con hardware che invalida selettivamente i blocchi (*snooping*). **Memoria virtuale**: se la DMA usa indirizzi fisici, un trasferimento non deve attraversare il confine di una pagina, perché pagine virtuali consecutive possono non essere consecutive in memoria fisica; se usa indirizzi virtuali, il controllore deve tradurli. In entrambi i casi il sistema operativo non deve spostare le pagine coinvolte durante il trasferimento.

Esempi ed esercizi svolti

> **Aggiunta in revisione · fonte: esercizi nuovi sul modello delle slide (Gerarchia di memoria, Input-Output); sequenze simulate con `strumenti/cache.py`, conti controllati con `strumenti/verifica_ampliamento.py` (ADC-05)**

Esercizio 1. Dimensionare una cache. Indirizzi di 32 bit, 64 KB di dati, blocchi di 32 byte. Campi dell'indirizzo e bit totali, a corrispondenza diretta e a 4 vie.

Blocchi: $64\,\mathrm{KB}/32\,\mathrm{B} = 2048 = 2^{11}$. Offset: 5 bit. *Diretta*: indice 11 bit, tag $32 - 11 - 5 = 16$ bit. Ogni blocco ha 256 bit di dati, 16 di tag e 1 di validità: $2^{11} \times 273 = 559\,104$ bit, cioè $68{,}25$ KB (il 6,6% in più dei dati). *4 vie*: $2048/4 = 512 = 2^9$ insiemi, indice 9 bit, tag 18 bit; $2^{11} \times 275 = 563\,200$ bit, più i bit per l'LRU. Più vie, più bit di tag e più comparatori (4 invece di 1).

Esercizio 2. Una sequenza di accessi in tre cache. Cache di 8 blocchi da una parola. Indirizzi in byte: 0, 8, 32, 0, 40, 8, 64, 32. Hit e miss con corrispondenza diretta, a 2 vie (LRU) e completamente associativa (LRU).

Gli indirizzi delle parole sono 0, 2, 8, 0, 10, 2, 16, 8.

Diretta (posizione = parola mod 8): 0, 2, 0, 0, 2, 2, 0, 0. Le parole 0, 8 e 16 si contendono la posizione 0, le parole 2 e 10 la posizione 2: ogni accesso caccia il blocco che servirà dopo. Otto miss, nessun hit.

2 vie, 4 insiemi (insieme = parola mod 4): tutte le parole vanno negli insiemi 0 e 2.

Accesso	0	8	32	0	40	8	64	32
Insieme	0	2	0	0	2	2	0	0
Esito	M	M	M	H	M	H	M	M
Insieme 0 dopo	0	0	0, 32	32, 0	32, 0	32, 0	0, 64	64, 32

(nell'ultima riga il blocco più recente è a destra). L'accesso a 64 sostituisce 32, il meno usato di recente, che serve subito dopo: due hit.

Completamente associativa: gli indirizzi diversi sono 5 e la cache ne contiene 8, quindi gli unici miss sono i 5 obbligatori: tre hit (il secondo 0, il secondo 8, il secondo 32).

Classificazione: nella cache diretta 5 miss sono obbligatori e 3 di conflitto; nessuno è di capacità, perché la cache completamente associativa della stessa dimensione non li ha.

Esercizio 3. La località spaziale. Cache diretta di 4 blocchi da 16 byte; un programma legge in ordine le 16 parole di un array che parte dall'indirizzo 0.

Ogni blocco contiene 4 parole. La prima lettura di ogni blocco è un miss, le altre tre sono hit: M H H H M H H H … , 4 miss e 12 hit, hit rate del 75%. Con blocchi di una parola sarebbero 16 miss.

Esercizio 4. Tempo medio di accesso con uno e due livelli. L1: hit time 1 ciclo, miss rate 5%; memoria: 100 cicli. Poi si aggiunge una L2 con hit time 10 cicli e miss rate locale del 40%.

Un livello: $\text{AMAT} = 1 + 0{,}05 \times 100 = 6$ cicli. Due livelli: la miss penalty della L1 diventa $10 + 0{,}4 \times 100 = 50$ cicli, e $\text{AMAT} = 1 + 0{,}05 \times 50 = 3{,}5$ cicli. Il miss rate globale della L2 è $0{,}05 \times 0{,}4 = 2\%$.

Esercizio 5. Il CPI con gli stalli di memoria. CPI di base 1,5; miss rate della cache istruzioni 2%, della cache dati 5%; il 30% delle istruzioni sono `lw` o `sw`; miss penalty 120 cicli. Poi si aggiunge una L2 con hit time 20 cicli e miss rate globale dell'1% (per istruzioni e dati).

Senza L2: stalli per istruzione $0{,}02 \times 120 + 0{,}3 \times 0{,}05 \times 120 = 2{,}4 + 1{,}8 = 4{,}2$, e $\text{CPI} = 5{,}7$. Il processore passa quasi tre quarti del tempo ad aspettare la memoria. Con la L2: ogni miss della L1 costa 20 cicli, e quelli che mancano anche la L2 altri 120:

$$\text{CPI} = 1{,}5 + (0{,}02 + 0{,}3 \times 0{,}05) \times 20 + (0{,}01 + 0{,}3 \times 0{,}01) \times 120 = 1{,}5 + 0{,}7 + 1{,}56 = 3{,}76.$$

L'accelerazione dovuta alla L2 è $5{,}7 / 3{,}76 \approx 1{,}52$.

Esercizio 6. La miss penalty e la DRAM. Blocchi di 8 parole, bus largo una parola: 1 ciclo per l'indirizzo, 25 cicli per un accesso alla DRAM, 6 cicli per le parole successive nella stessa riga, 1 ciclo per trasferire una parola.

a. Ogni parola con un accesso completo: $1 + 8 \times 25 + 1 = 202$ cicli; banda $32/202 \approx 0{,}16$ byte per ciclo. (b) Parole nella stessa riga: $1 + 25 + 7 \times 6 + 1 = 69$ cicli; banda $32/69 \approx 0{,}46$. (c) Memoria interallacciata in 8 banchi: $1 + 25 + 8 \times 1 = 34$ cicli; banda $32/34 \approx 0{,}94$. L'interallacciamento divide la miss penalty per sei.

Esercizio 7. Write-through o write-back? Un ciclo esegue 100 volte `sw` sulla stessa parola, poi il blocco viene sostituito. Quante scritture arrivano alla memoria?

Con il write-through ogni `sw` scrive anche in memoria: 100 scritture di una parola (smaltite dal write buffer, se tiene il passo). Con il write-back le 100 scritture restano in cache e il blocco, sporco, viene riscritto una sola volta alla sostituzione. Al contrario, per dati scritti una sola volta e mai riletti il write-back non risparmia nulla.

Esercizio 8. Indirizzi virtuali. Indirizzi virtuali di 32 bit, pagine di 4 KB, voci della tabella delle pagine di 4 byte. Quanto è grande la tabella di un processo?

Offset: $4\,\text{KB} = 2^{12}$, 12 bit. Numero di pagina virtuale: 20 bit, cioè 2^{20} pagine. Tabella: $2^{20} \times 4\,\text{B} = 4$ MB per processo; è per questo che nei sistemi reali le tabelle sono a più livelli e si caricano solo le parti usate.

Esercizio 9. Il costo del polling e della DMA. Processore a 1 GHz; ogni interrogazione costa 400 cicli. (a) Un mouse interrogato 30 volte al secondo. (b) Un disco che trasferisce 4 MB/s a blocchi di 16 byte, interrogato ogni volta che può avere un dato. (c) Lo stesso disco con DMA a blocchi di 16 KB, con 1000 cicli per programmare il trasferimento e 500 per l'interruzione finale (1 MB = 10^6 byte, 1 KB = 10^3 byte).

a. $30 \times 400 = 12\,000$ cicli al secondo: lo $0{,}0012\%$ del processore, trascurabile. (b) $4 \cdot 10^6 / 16 = 250\,000$ interrogazioni al secondo, $250\,000 \times 400 = 10^8$ cicli: il 10% del processore, solo per il disco. (c) $4 \cdot 10^6 / 16\,000 = 250$ trasferimenti al secondo, $250 \times 1500 = 375\,000$ cicli: lo $0{,}0375\%$.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; soluzioni controllate con `strumenti/cache.py` e `strumenti/verifica_ampliamento.py` (ADC-05)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. Da un indirizzo ai campi. Nella cache diretta dell'esercizio 1, dove va l'indirizzo `0x1234ABCD`?

Passo 1: i campi. Tag 16 bit, indice 11, offset 5.

Passo 2: in binario la parte bassa. `0xABCD` $= 1010\,1011\,1100\,1101$.

Passo 3: offset. I 5 bit bassi, $01101 = 13$: è il byte 1 della parola 3 del blocco.

Passo 4: indice. I bit da 5 a 15, $10101011110_2 = 1374$ (`0x55E`).

Passo 5: tag. I 16 bit alti, `0x1234`. Il controllore guarda la posizione 1374, controlla il bit di validità e confronta il tag memorizzato con `0x1234`.

Guidato 2. La stessa cache con blocchi più grandi. Cache diretta da 32 byte. Accessi (indirizzi in byte): 0, 4, 16, 64, 68, 0, 20. Blocchi da 4 byte (8 blocchi) o da 16 byte (2 blocchi)?

Passo 1: blocchi da 4 byte. Parole 0, 1, 4, 16, 17, 0, 5; posizioni (mod 8) 0, 1, 4, 0, 1, 0, 5. Il 64 caccia lo 0, il 68 caccia il 4, il secondo 0 caccia il 64: tutti miss, 7 su 7.

Passo 2: blocchi da 16 byte. Blocchi di memoria 0, 0, 1, 4, 4, 0, 1; posizioni (mod 2) 0, 0, 1, 0, 0, 0, 1. Esiti: M, H (stesso blocco), M, M (il blocco 4 caccia lo 0), H, M (lo 0 caccia il 4), H (il blocco 1 è ancora lì). 4 miss e 3 hit.

Passo 3: la morale. Con blocchi più grandi la località spaziale (0 e 4, 64 e 68, 16 e 20) diventa hit; ma con soli 2 blocchi i conflitti fra 0 e 64 restano.

Guidato 3. Un processore più veloce con la stessa memoria. Il processore dell'esercizio 5 (senza L2) raddoppia la frequenza di clock; la memoria resta uguale. Di quanto migliora?

Passo 1: la miss penalty in cicli. La memoria impiega lo stesso tempo, quindi i cicli raddoppiano: 240.

Passo 2: il nuovo CPI. $1{,}5 + (0{,}02 + 0{,}3 \times 0{,}05) \times 240 = 1{,}5 + 8{,}4 = 9{,}9$.

Passo 3: il confronto. Tempo per istruzione: prima $5{,}7\,T$, ora $9{,}9 \times T/2 = 4{,}95\,T$. L'accelerazione è $5{,}7/4{,}95 \approx 1{,}15$, non 2: è la legge di Amdahl, perché la parte dovuta alla memoria non è accelerata.

Riepilogo

La gerarchia di memoria sfrutta la località temporale e spaziale per dare l'illusione di una memoria grande come l'ultimo livello e veloce quasi come il primo: SRAM per le cache, DRAM per la memoria principale, dischi per la memoria secondaria. La cache a corrispondenza diretta mette ogni blocco in una sola posizione, data dall'indice; tag e bit di validità dicono se il blocco è presente. Le scritture si gestiscono con write-through e write buffer, o con write-back e dirty bit; nei miss in scrittura si sceglie fra write allocate e no-write allocate. I miss sono obbligatori, di capacità o di conflitto; i conflitti si riducono con le cache associative a n vie, che richiedono più comparatori e una politica di sostituzione come LRU. Le prestazioni si misurano con il tempo medio di accesso e con gli stalli aggiunti al CPI; le cache a più livelli riducono la miss penalty, e l'organizzazione della DRAM e del bus (stessa riga, interallacciamento) la banda. La memoria virtuale traduce gli indirizzi a pagine con la tabella delle pagine e il TLB. L'I/O usa bus sincroni o asincroni, I/O mappato in memoria, polling, interruzioni e DMA, con i problemi di coerenza fra DMA e cache.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (ADC-05)**

Indirizzi e dimensioni

$$\text{offset} = \log_2(\text{byte per blocco}), \quad \text{indice} = \log_2(\text{insiemi}), \quad \text{tag} = 32 - \text{indice} - \text{offset}, \quad \text{insiemi} = \frac{\text{blocchi}}{\text{vie}}.$$

$$\text{posizione} = \text{blocco} \bmod \text{insiemi}, \qquad \text{bit totali} = \text{blocchi} \times (\text{bit del blocco} + \text{tag} + 1\ [+1\ \text{dirty}]).$$

Prestazioni

$$\text{AMAT} = t_{hit} + m \cdot P, \qquad \text{AMAT}_2 = t_{L1} + m_{L1}(t_{L2} + m_{L2,loc} P_{mem}), \qquad m_{L2,glob} = m_{L1} \cdot m_{L2,loc}.$$

$$\text{CPI} = \text{CPI}_{base} + m_I \cdot P + f_{ld/st} \cdot m_D \cdot P.$$

Miss penalty con bus da una parola e blocchi di k parole: $1 + k \cdot t_{DRAM} + 1$; stessa riga: $1 + t_{DRAM} + (k-1)t_{col} + 1$; k banchi: $1 + t_{DRAM} + k$. Banda = byte del blocco / miss penalty.

Politiche

Scelta	Opzioni
Posizionamento	diretta, n vie, completamente associativa

Scelta	Opzioni
Sostituzione	LRU, NRU, casuale
Scrittura (hit)	write-through (+ write buffer), write-back (+ dirty bit)
Scrittura (miss)	write allocate, no-write allocate
Miss	obbligatori, di capacità, di conflitto
I/O	polling, interruzioni, DMA

Termini per l’orale

Termine	In una riga
Località temporale, spaziale	riuso dello stesso dato; uso dei dati vicini
Blocco (linea)	unità di trasferimento fra due livelli
Hit time, miss penalty	tempo di un hit; costo di un miss
Tag, indice, offset	bit alti di confronto; posizione; parola e byte nel blocco
Bit di validità, dirty bit	posizione piena; blocco modificato
Write buffer	coda delle scritture verso la memoria
Miss rate locale, globale	rispetto agli accessi al livello; rispetto a tutti
Interallacciamento	parole consecutive in banchi diversi
TLB	cache delle traduzioni degli indirizzi
I/O mappato in memoria	registri dei dispositivi come indirizzi di memoria
DMA	trasferimento di blocchi senza il processore
Snooping	controllore che osserva il bus e invalida i blocchi

Domande che tornano spesso

- Perché serve una gerarchia di memoria e perché funziona? SRAM e DRAM.
- Come si trova un dato in una cache diretta, associativa a insiemi e completamente associativa? Calcolo di tag, indice e offset.
- Politiche di scrittura e di sostituzione; le tre cause dei miss.
- Formula dell’AMAT e del CPI con gli stalli; perché le cache a più livelli?
- Come si riduce la miss penalty con l’organizzazione della DRAM?
- Polling, interruzioni e DMA; i problemi della DMA con cache e memoria virtuale.